

# *W4111 – Introduction to Databases*

## *Lecture 11: Module II (4), NoSQL (4)*



# *Contents*

# Contents

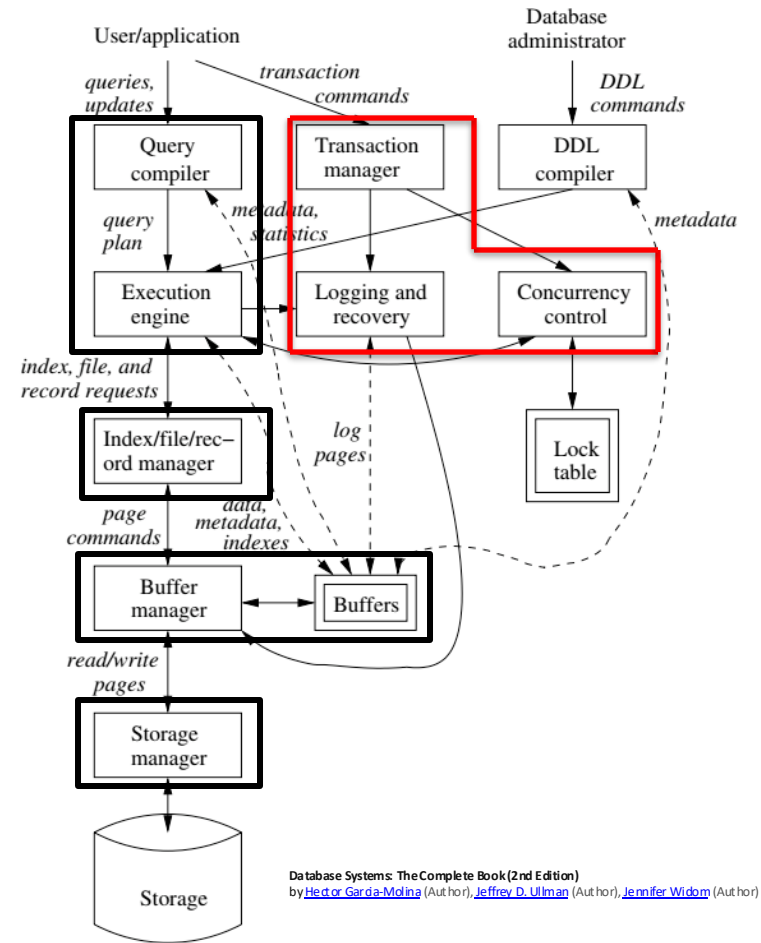
- Database design considerations:
  - Normalization simpler explanation and some worked examples.
  - Wide-flat versus normalization tradeoff and example.
- Module II – Transaction and recovery:
  - Concepts
  - Atomicity and Durability
  - Isolation
- Scalability and Availability
- Data Enabled Decision Support
  - Overview
  - Enterprise Information Integration, Extract Transform Load, Data Lake, Data Warehouse
  - OLAP
- HW 3a, 3b and Project
  - Overview
  - REST
  - Some examples
- Reference/Backup Material

# *Module II Reminder*

# Data Management

## Previously

- Load/save things quickly.
- Storage Mgmt. (cont)
- Access data quickly.
- Find things quickly (cont).
- Query processing, e.g. transform
  - Declarative language to
  - Procedural, functional, execution control
- Today's topics:
  - Transactions
  - Recovery



# *Database Design Considerations*

# *Normalization Simplified*

# Ferguson's Laws of Teaching Databases

- **First Law:** “There is no database concept so simple that database textbooks and course material cannot make the concept baffling and incomprehensible.”
- **Second Law:** “No matter how baffling and incomprehensible a textbook or course material has made a simple concept, the course material will enhance the bewilderment by using unintelligible, confusing notation and quasi-math.”
- My girlfriend saw this slide and added a 3<sup>rd</sup> Law.
- **Third Law:** “The grumpy professor will inevitably make the concept terrifying in addition to baffling and incomprehensible.”



# Goals of Normalization

- Let  $R$  be a relation scheme with a set  $F$  of functional dependencies.
- Decide whether a relation scheme  $R$  is in “good” form.
- In the case that a relation scheme  $R$  is not in “good” form, need to decompose it into a set of relation scheme  $\{R_1, R_2, \dots, R_n\}$  such that:
  - Each relation scheme is in good form
  - The decomposition is a lossless decomposition
  - Preferably, the decomposition should be dependency preserving.
- DFF says, in a nutshell,
  - There are a few simple patterns to look for and fix.
  - Use common sense.



# Denormalization for Performance

- ❑ May want to use non-normalized schema for performance
- ❑ For example, displaying *prereqs* along with *course\_id*, and *title* requires join of *course* with *prereq*
- ❑ Alternative 1: Use denormalized relation containing attributes of *course* as well as *prereq* with all above attributes
  - ❑ faster lookup
  - ❑ extra space and extra execution time for updates
  - ❑ extra coding work for programmer and possibility of error in extra code
- ❑ Alternative 2: use a materialized view defined a *course* ⋈ *prereq*
  - ❑ Benefits and drawbacks same as above, except no extra coding work for programmer and avoids possible errors

# Simple Explanations

There are many good examples and tutorials on the web, e.g.

<https://www.datacamp.com/tutorial/normalization-in-sql>

## Types of Normalization in SQL



### 1NF

First Normal Form

- Ensures that each column contains only atomic values



### 2NF

Second Normal Form

- Eliminates partial dependencies.



### 3NF

Third Normal Form

- Eliminates transitive dependencies.



### BCNF

Boyce-Codd Normal Form

- Strict version of 3NF that addresses additional anomalies.



### 4NF

Fourth Normal Form

- Deals with multi-valued dependencies



### 5NF

Fifth Normal Form

- Addresses join dependencies

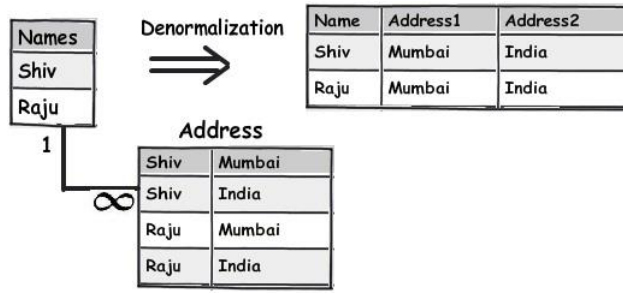
# Simple Explanations

There are a lot of good, simple, short explanations and tutorials on the web.

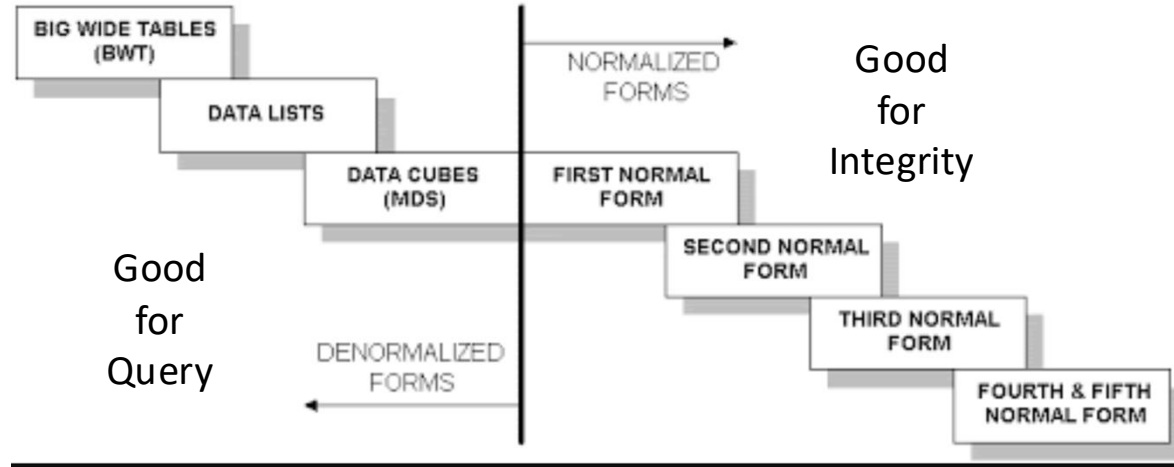
- First Normal Form (1NF)
  - This normalization level ensures that each column in your data contains only atomic values. Atomic values in this context means that each entry in a column is indivisible. It is like saying that each cell in a spreadsheet should hold just one piece of information. 1NF ensures atomicity of data, with each column cell containing only a single value and each column having unique names.
- Second Normal Form (2NF)
  - Eliminates partial dependencies by ensuring that non-key attributes depend only on the primary key. What this means, in essence, is that there should be a direct relationship between each column and the primary key, and not between other columns.
- Third Normal Form (3NF)
  - Removes transitive dependencies by ensuring that non-key attributes depend only on the primary key. This level of normalization builds on 2NF.
- Boyce-Codd Normal Form (BCNF)
  - This is a more strict version of 3NF that addresses additional anomalies. At this normalization level, every determinant is a candidate key.
- Fourth Normal Form (4NF)
  - This is a normalization level that builds on BCNF by dealing with multi-valued dependencies.
- Fifth Normal Form (5NF)
  - 5NF is the highest normalization level that addresses join dependencies. It is used in specific scenarios to further minimize redundancy by breaking a table into smaller tables.

# *Wide-Flat versus Normalization*

# Wide Flat Tables



- Improve query performance by precomputing and saving:
  - JOINS
  - Aggregation
  - Derived/computed columns
- One of the primary strength of the relational model is maintaining “integrity” when applications create, update and delete data. This relies on:
  - The core capabilities of the relational model, e.g. constraints.
  - A well-design database (We will cover a formal definition – “normalization” in more detail later.
- Data models that are well designed for integrity are very bad for read only analysis queries. We will build and analyze wide flat tables as part of the analysis tasks in HW3, HW4 as projects.

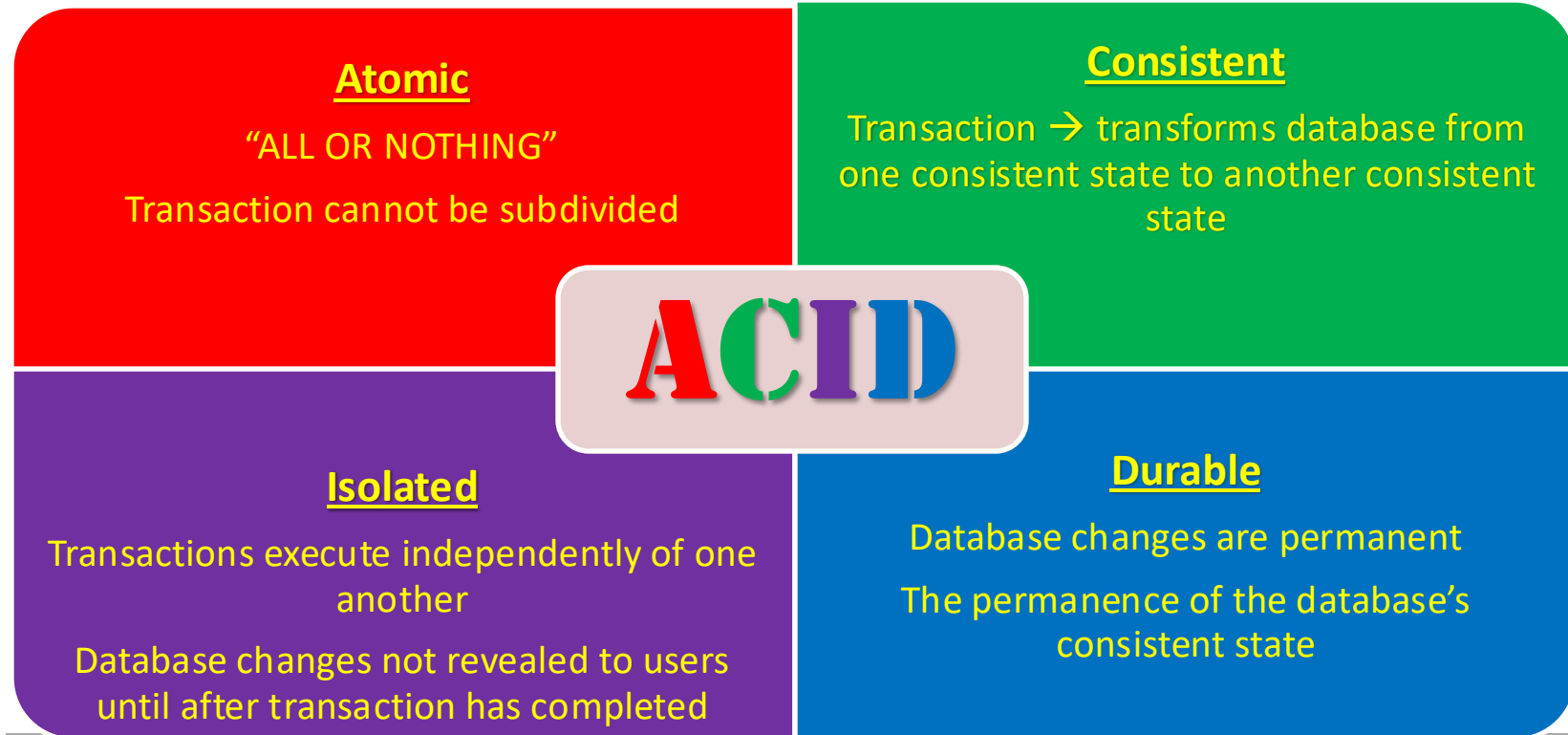


# *Transactions and Recovery*

# *Core Concepts*

# Core Transaction Concept is ACID Properties

<http://slideplayer.com/slide/9307681>



A *transaction* is a very small unit of a program and it may contain several low-level tasks. A transaction in a database system must maintain **Atomicity**, **Consistency**, **Isolation**, and **Durability** – commonly known as ACID properties – in order to ensure accuracy, completeness, and data integrity.

- **Atomicity** – This property states that a transaction must be treated as an atomic unit, that is, either all of its operations are executed or none. There must be no state in a database where a transaction is left partially completed. States should be defined either before the execution of the transaction or after the execution/abortion/failure of the transaction.
- **Consistency** – The database must remain in a consistent state after any transaction. No transaction should have any adverse effect on the data residing in the database. If the database was in a consistent state before the execution of a transaction, it must remain consistent after the execution of the transaction as well.
- **Durability** – The database should be durable enough to hold all its latest updates even if the system fails or restarts. If a transaction updates a chunk of data in a database and commits, then the database will hold the modified data. If a transaction commits but the system fails before the data could be written on to the disk, then that data will be updated once the system springs back into action.
- **Isolation** – In a database system where more than one transaction are being executed simultaneously and in parallel, the property of isolation states that all the transactions will be carried out and executed as if it is the only transaction in the system. No transaction will affect the existence of any other transaction.



# Transaction Concept

- A **transaction** is a *unit* of program execution that accesses and possibly updates various data items.
- E.g., transaction to transfer \$50 from account A to account B:

BEGIN TRANSACTION {

1. **read**(A)

2.  $A := A - 50$

3. **write**(A)

4. **read**(B)

5.  $B := B + 50$

6. **write**(B)

COMMIT or ROLLBACK }

- Two main issues to deal with:
  - Failures of various kinds, such as hardware failures and system crashes
  - Concurrent execution of multiple transactions

BEGIN TRANSACTION {

1. **read**(A)

2.  $A := A - 50$

3. **write**(A)

4. **read**(B)

5.  $B := B + 50$

6. **write**(B)

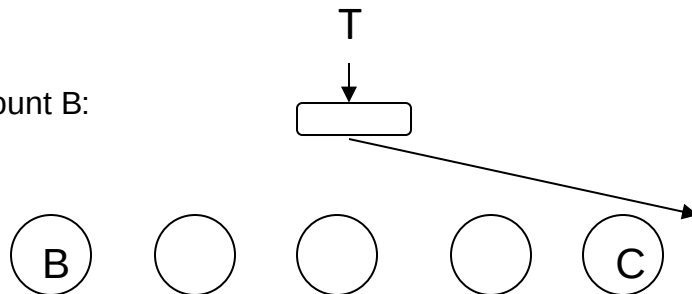
COMMIT or ROLLBACK }



# Example of Fund Transfer

- Transaction to transfer \$50 from account A to account B:

1. **read**(A)
2.  $A := A - 50$
3. **write**(A)
4. **read**(B)
5.  $B := B + 50$
6. **write**(B)



- Atomicity requirement**

- If the transaction fails after step 3 and before step 6, money will be “lost” leading to an inconsistent database state
  - Failure could be due to software or hardware
- The system should ensure that updates of a partially executed transaction are not reflected in the database

- Durability requirement** — once the user has been notified that the transaction has completed (i.e., the transfer of the \$50 has taken place), the updates to the database by the transaction must persist even if there are software or hardware failures.



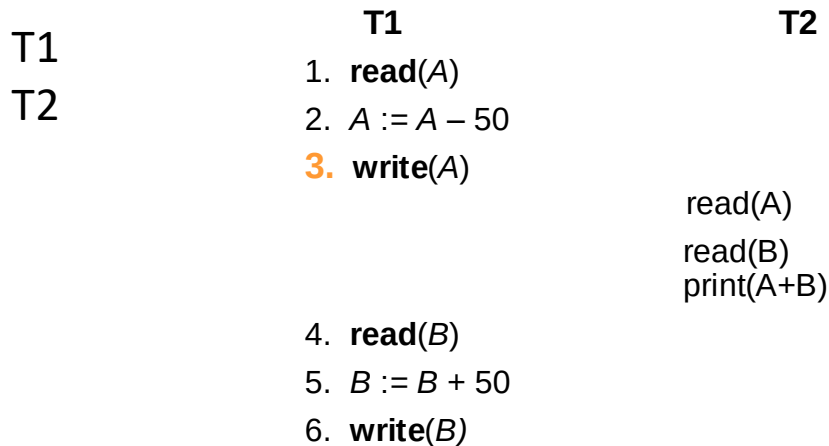
## Example of Fund Transfer (Cont.)

- **Consistency requirement** in above example:
  - The sum of A and B is unchanged by the execution of the transaction
- In general, consistency requirements include
  - Explicitly specified integrity constraints such as primary keys and foreign keys
  - Implicit integrity constraints
    - e.g., sum of balances of all accounts, minus sum of loan amounts must equal value of cash-in-hand
  - A transaction must see a consistent database.
  - During transaction execution the database may be temporarily inconsistent.
  - When the transaction completes successfully the database must be consistent
    - Erroneous transaction logic can lead to inconsistency



## Example of Fund Transfer (Cont.)

- T1, T2
- **Isolation requirement** — if between steps 3 and 6, another transaction T2 is allowed to access the partially updated database, it will see an inconsistent database (the sum  $A + B$  will be less than it should be).



- Isolation can be ensured trivially by running transactions **serially**
  - That is, one after the other.
- However, executing multiple transactions concurrently has significant benefits, as we will see later.



# ACID Properties

A **transaction** is a unit of program execution that accesses and possibly updates various data items. To preserve the integrity of data the database system must ensure:

- **Atomicity.** Either all operations of the transaction are properly reflected in the database or none are.
- **Consistency.** Execution of a transaction in isolation preserves the consistency of the database.
- **Isolation.** Although multiple transactions may execute concurrently, each transaction must be unaware of other concurrently executing transactions. Intermediate transaction results must be hidden from other concurrently executed transactions.
  - That is, for every pair of transactions  $T_i$  and  $T_j$ , it appears to  $T_i$  that either  $T_j$  finished execution before  $T_i$  started, or  $T_j$  started execution after  $T_i$  finished.
- **Durability.** After a transaction completes successfully, the changes it has made to the database persist, even if there are system failures.



# Transaction State

- **Active** – the initial state; the transaction stays in this state while it is executing
- **Partially committed** – after the final statement has been executed.
- **Failed** -- after the discovery that normal execution can no longer proceed.
- **Aborted** – after the transaction has been rolled back and the database restored to its state prior to the start of the transaction. Two options after it has been aborted:
  - Restart the transaction
    - Can be done only if no internal logical error
  - Kill the transaction
- **Committed** – after successful completion.

# Atomicity Durability

# Atomicity

A transaction is a logical unit of work that must be either entirely completed or entirely undone. (All writes happen or none of them happen)

OK. What does this mean? Consider some pseudo-code

```
def transfer(source_acct_id, target_acct_id, amount)
```

1. Check that both accounts exist.
2. IF *is\_checking\_account(source\_acct\_id)*
  1. Check that  $(\text{source\_acct.balance} - \text{amount}) > \text{source\_account.overdraft\_limit}$
3. ELSE
  1. Check that  $(\text{source\_count.balance} - \text{amount}) > \text{source\_account.minimum\_balance}$
4. Update source account
5. Update target account.
6. INSERT a record into transfer tracking table.

# Atomicity

A transaction is a logical unit of work that must be either entirely completed or entirely undone. (All writes happen or none of them happen)

OK. What does this mean? Consider some pseudo-code

```
def transfer(source_acct_id, target_acct_id, amount)
```

1. Check that both accounts exist.
2. IF *is\_checking\_account(source\_acct\_id)*
  1. Check that  $(\text{source\_acct.balance} - \text{amount}) > \text{source\_acc}$
3. ELSE
  1. Check that  $(\text{source\_count.balance} - \text{amount}) > \text{source\_ac}$
4. Update source account
5. Update target account.
6. INSERT a record into transfer tracking table.

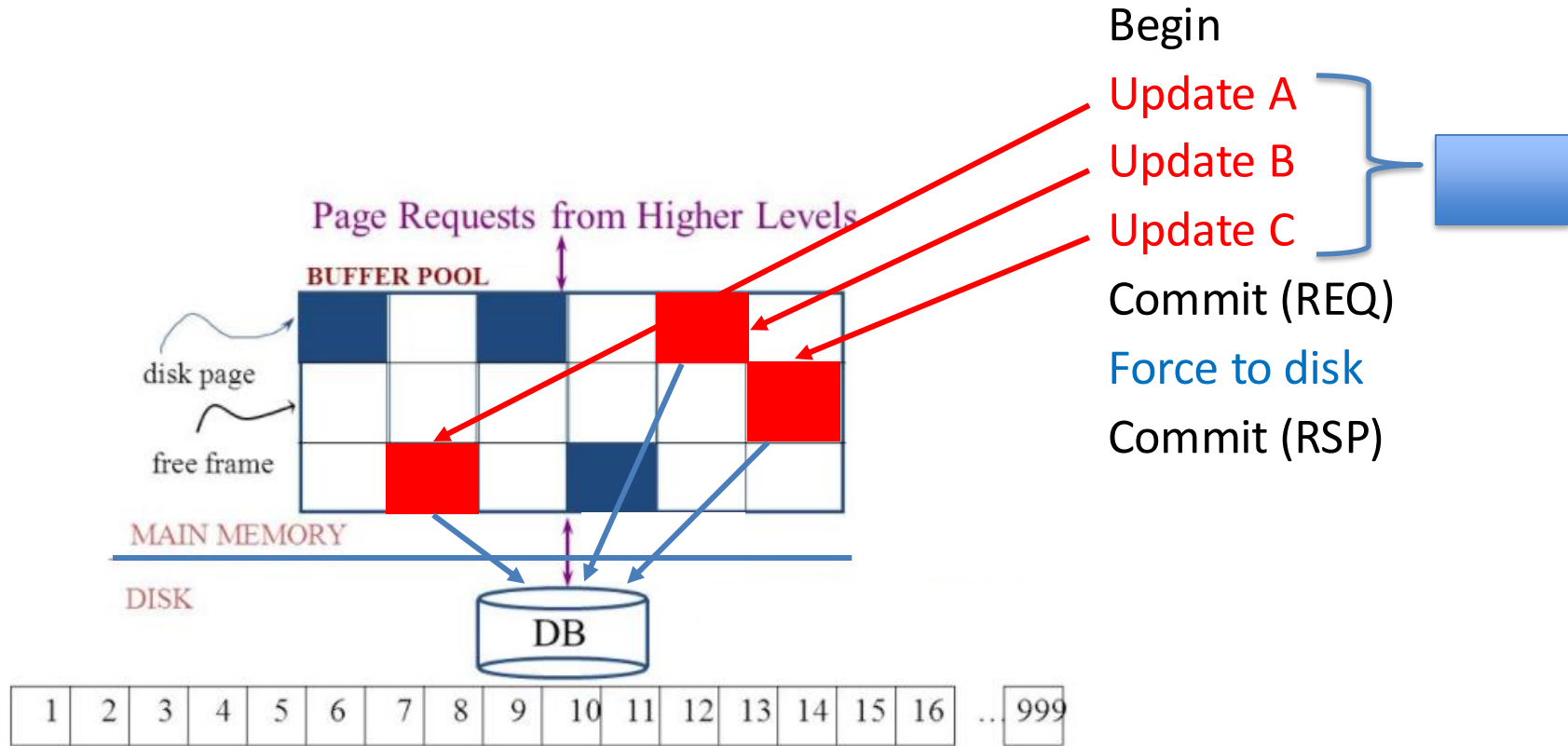


Computer  
or  
DB Server fails.

# Atomicity

- Transaction programs and databases are fast (milliseconds).  
What are the chances of the failure occurring in the wrong spot?
- Well, that doesn't really matter. If it happens,
  - Someone lost money and
  - There is no record of it. Someone is going to be very upset.
- Even a small server can have thousands of concurrent transactions →
  - There will be corruptions because some transaction will be in the wrong place at the wrong time.
  - Unless we do something in the DBMS
  - Because HW and software inevitably fail
  - And sadly, SW is especially prone to failure when under load

# Simplistic Approach



# Simplistic Approach

There are several problems with the simplistic approach.

1. The approach does not solve the problem
  1. Some writes might succeed.
  2. Some might be interrupted by the failure, or require retry.
2. Writes may be random and scattered. N updates might

1. Change a few bytes in N data frames
2. A few bytes in M index frames

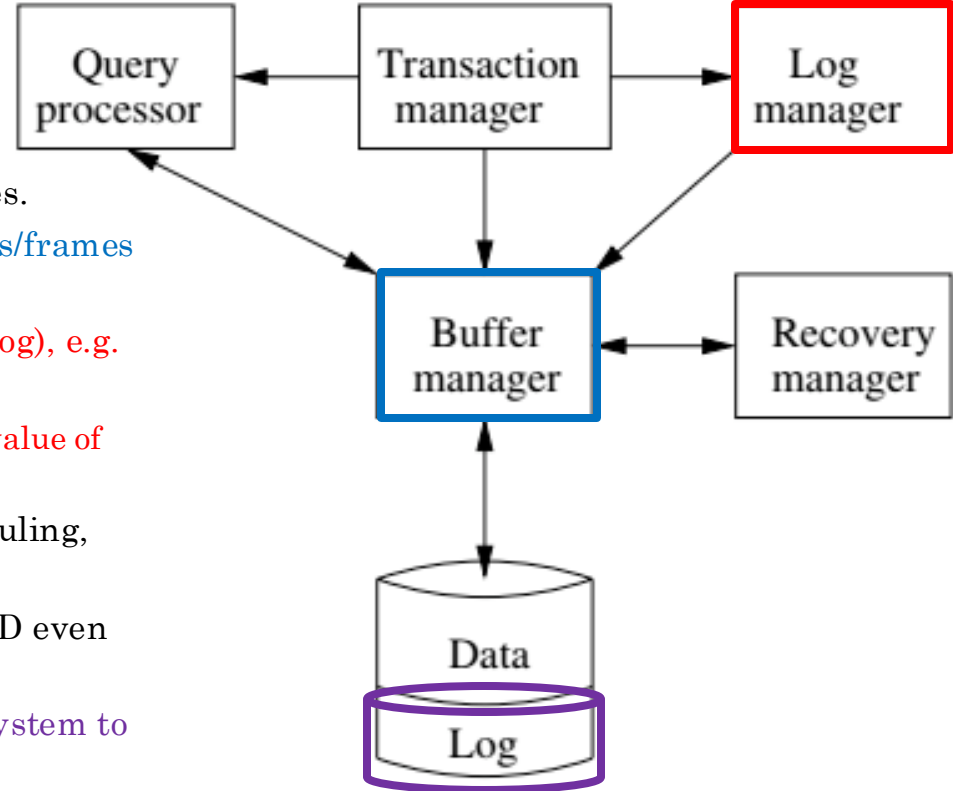
Transaction rate becomes bottlenecked by write I/O rate, even though a relative small number of bytes change/transaction.

3. Written frames must be held in memory.
  1. Lots of transactions
  2. Randomly writing small pieces of lots of frames.
  3. Consumes lots of memory with pinned pages.
  4. Degrades the performance and optimization of the buffer.
    1. The optimal buffer replacement policy wants to hold frames that will be reused.
    2. Not frames that have been touched and never reused.

# DBMS ACID Implementation

## Implementation Subsystems

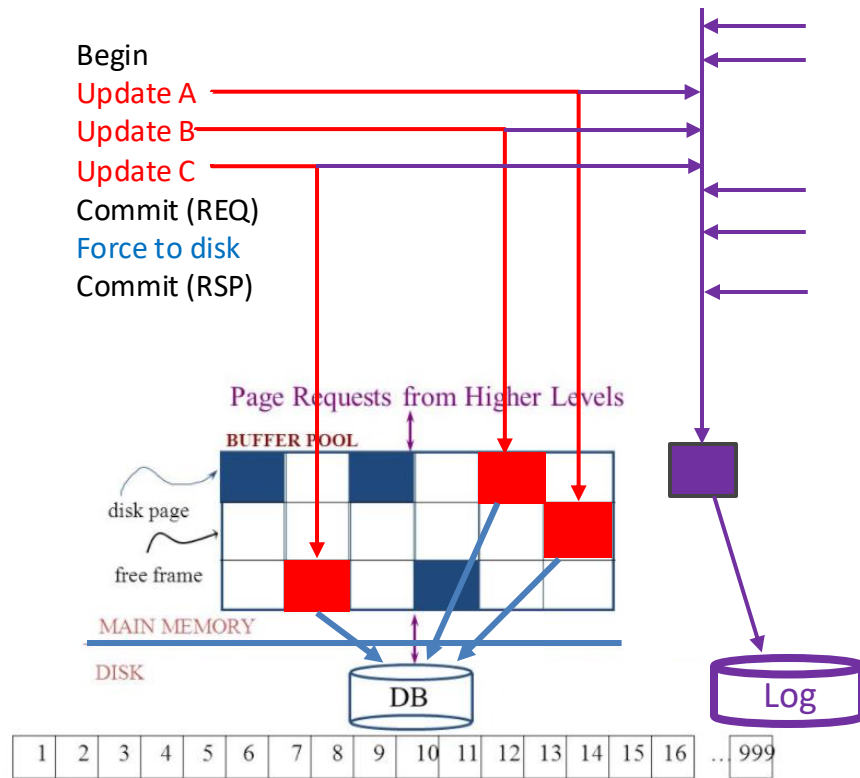
- *Query processor* schedules and executes queries.
- *Buffer manager* controls reading/writing blocks/frames to/from disk.
- *Log manager* journals/records events to disk (log), e.g.
  - Transaction start/commit/abort
  - Transaction update of a block and previous value of data.
- *Transaction manager* coordinates query scheduling, buffer read/write and logging to ensure ACID.
- *Recovery manager* processes log to ensure ACID even after transaction or system failures.
- *Log* is a special type of block file used by the system to optimize performance and ensure ACID.



The DBMS logs every transaction event

- *Log Sequence Number (LSN)*: A unique ID for a log record.
- *Prev LSN*: A link to their last log record.
- *Transaction ID number*.
- *Type*: Describes the type of database log record.
  - **Update Log Record**
    - *PageID*: A reference to the Page ID of the modified page.
    - *Length and Offset*: Length in bytes and offset of the page are usually included.
    - *Before and After Images* of records.
  - **Compensation Log Record**
  - **Commit Record**
  - **Abort Record Checkpoint Record**
  - **Completion Record** notes that all work has been done for this particular transaction.

# Write Ahead Logging



## DBMS (Redo processing)

- Write log events from all transactions into a single log stream.
- Multiple events per page
- Forces (writes) log record on COMMIT/ABORT
  - Single block I/O records many updates
  - Versus multiple block I/Os, each recording a single change.
  - All of a transaction's updates recorded in one I/O versus many.
- If there is a failure
  - DBMS sequentially reads log.
  - Applies changes to modified pages that were not saved to disk.
  - Then resumes normal processing.

# Write Ahead Logging

- Force every write to disk?
  - Poor response time.
  - But provides durability.
- Steal buffer-pool frames from uncommitted transactions?
  - If not, poor performance/caching performance
  - If yes, how can we ensure atomicity?  
Uncommitted updates on disk

|          | No Steal | Steal   |
|----------|----------|---------|
| Force    | Trivial  |         |
| No Force |          | Desired |

## DBMS (Undo processing)

- Enable steal policy to improve cache performance by
  - Avoiding lots of pinned pages
  - Unlikely to be reused soon.
- Before stealing
  - Force log record to disk.
  - Update log entry has data record
    - Before image
    - After image
- If there is a failure
  - DBMS sequentially reads log.
  - Undoes changes to
    - modified pages, uncommitted pages
    - That were saved to disk.
  - Then resumes normal processing.

ARIES recovery involves three passes

1. Analysis pass:

- Determine which transactions to undo
- Determine which pages were dirty (disk version not up to date) at time of
- RedoLSN: LSN from which redo should start

2. Redo pass:

- Repeats history, redoing all actions from RedoLSN (updated committed but not written changes to pages)
- RecLSN and PageLSNs are used to avoid redoing actions already reflected on page

3. Undo pass:

- Rolls back all incomplete transactions (with uncommitted pages written to disk).
- Transactions whose abort was complete earlier are not undone

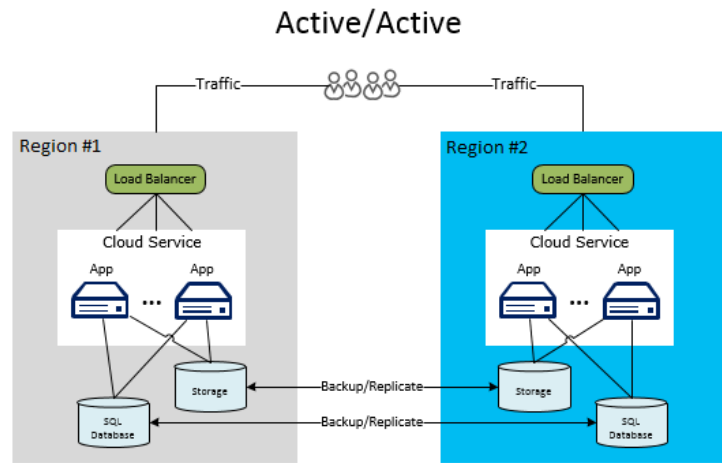
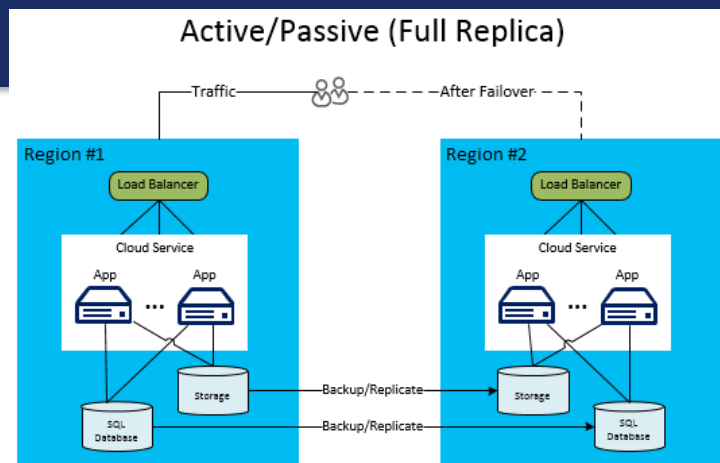
# Durability

- Write changes to disk trivially achieves durability.
- DBMS engine uses write-ahead-logging to
  - Achieve durability
  - But with better performance through more efficient caching and I/O.
- Well, disks fail. How is that durable.
  - RAID and other solutions.
  - Disk subsystems, including entire RAID device, fail →
    - Duplex writes
    - To independent disk subsystems.
- Well, there are earthquakes, floods, etc.



# Availability and Replication

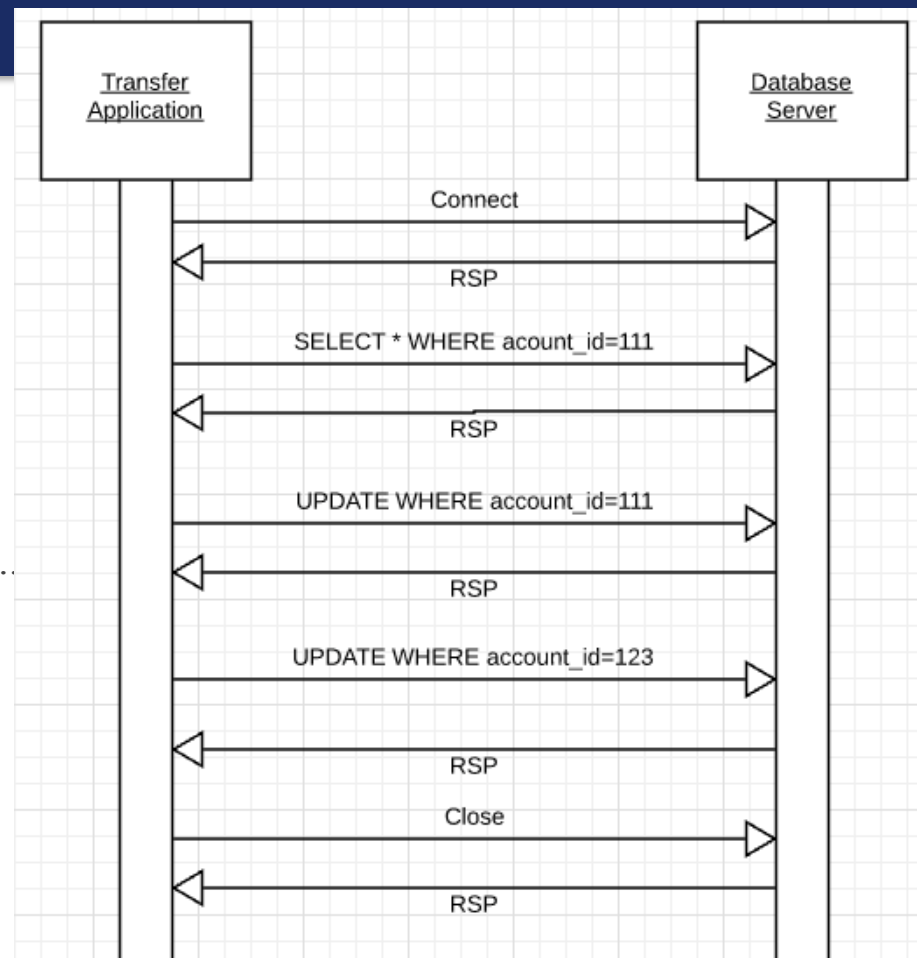
- There are two basic patterns
  - Active/Passive
    - All requests go to *master* during normal processing.
    - Updates are transactionally queued for processing at passive backup.
    - Failure of *master*
      - Routes subsequent requests to *backup*.
      - Backup must process and commit updates before accepting requests.
  - Active/Active
    - Both environments process requests.
    - Some form of distributed transaction commit required to synchronize updates on both copies.
- Multi-system communication to guarantee consistency is the foundation for tradeoffs in CAP.
  - The system can be CAP if and only iff
  - There are never any partitions or system failures
  - Which is unrealistic in cloud/Internet systems.



# *Isolation*

# Isolation

- Transfer \$50 from
  - account\_id=111 to
  - account\_id=123
- Requires 3 SQL statements
  - SELECT from 111 to check balance  $\geq$  \$50
  - UPDATE account\_id=111
  - UPDATE account\_id=123
- There are some interesting scenarios
  - Two different programs read the balance (\$51)
  - And decide removing \$50 is OK.
- DB constraints can prevent the conflict from happening, but ...
  - There are more complex scenarios that constraints do not prevent.
  - Not ALL databases support constraints.
  - The “correct” execution should be that
    - One transaction responds “insufficient funds”
    - Before attempting transfer instead of after attempting.



# Isolation

- Consider two simultaneous transfer transactions T1 and T2.
- There are two equally **correct** executions
  1. T1 transfers, T2 responds “insufficient funds” and does not attempt transfer
  2. T2 transfers, T1 responds “insufficient funds” and does not attempt transfer
- Each correct simultaneous execution is equivalent to a serial (sequential) execution schedule
  - (1) Execute T1, Execute T2
  - (2) Execute T2, Execute T1
- NOTE:
  - We are focusing on correctness not
  - Fairness:
    - We do not care which transaction was actually submitted first.
    - And probably do not know due to networking, etc.

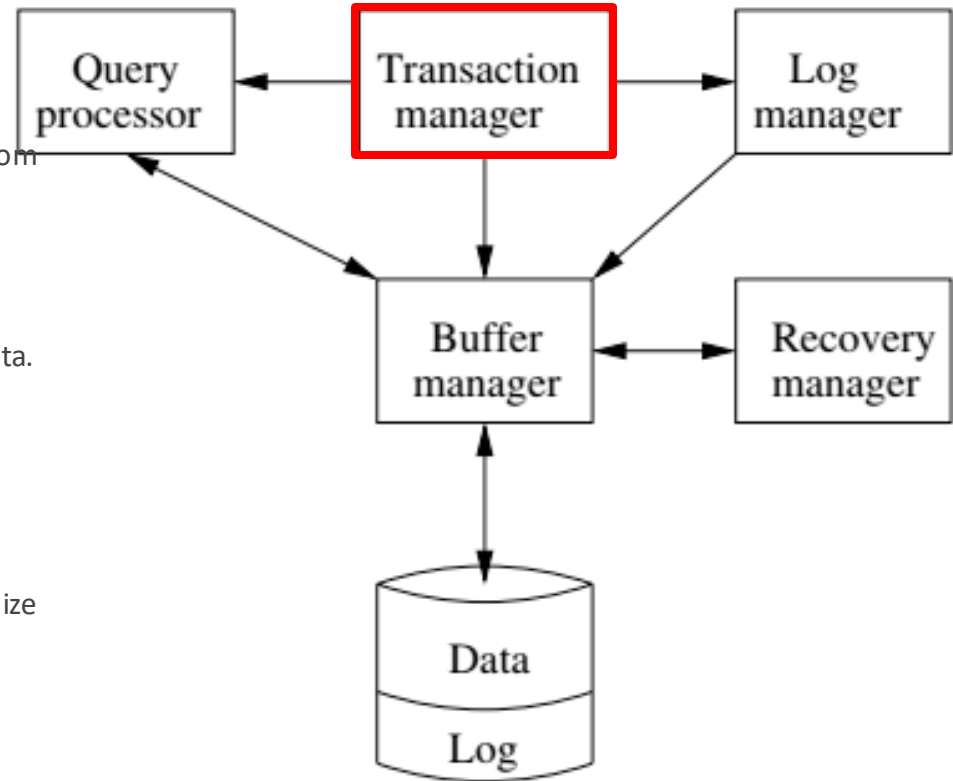
# Serializability

“In [concurrency control](#) of [databases](#),<sup>[1][2]</sup> [transaction processing](#) (transaction management), and various [transactional](#) applications (e.g., [transactional memory](#)<sup>[3]</sup> and [software transactional memory](#)), both centralized and [distributed](#), a transaction [schedule](#) is **serializable** if its outcome (e.g., the resulting database state) is equal to the outcome of its transactions executed serially, i.e. without overlapping in time. Transactions are normally executed concurrently (they overlap), since this is the most efficient way. Serializability is the major correctness criterion for concurrent transactions' executions. It is considered the highest level of [isolation](#) between [transactions](#), and plays an essential role in [concurrency control](#). As such it is supported in all general purpose database systems.”  
(<https://en.wikipedia.org/wiki/Serializability>)

# DBMS ACID Implementation

## Implementation Subsystems

- *Query processor* schedules and executes queries.
- *Buffer manager* controls reading/writing blocks/frames to/from disk.
- *Log manager* journals/records events to disk (log), e.g.
  - Transaction start/commit/abort
  - Transaction update of a block and previous value of data.
- *Transaction manager* coordinates query **scheduling**, buffer read/write and logging to ensure ACID.
- *Recovery manager* processes log to ensure ACID even after transaction or system failures.
- *Log* is a special type of block file used by the system to optimize performance and ensure ACID.



Garcia-Molina et al., p. 846

# Schedule

## 18.1.1 Schedules

A *schedule* is a sequence of the important actions taken by one or more transactions. When studying concurrency control, the important read and write actions take place in the main-memory buffers, not the disk. That is, a database element  $A$  that is brought to a buffer by some transaction  $T$  may be read or written in that buffer not only by  $T$  but by other transactions that access  $A$ .

| $T_1$           | $T_2$           |
|-----------------|-----------------|
| READ( $A, t$ )  | READ( $A, s$ )  |
| $t := t+100$    | $s := s*2$      |
| WRITE( $A, t$ ) | WRITE( $A, s$ ) |
| READ( $B, t$ )  | READ( $B, s$ )  |
| $t := t+100$    | $s := s*2$      |
| WRITE( $B, t$ ) | WRITE( $B, s$ ) |

Figure 18.2: Two transactions

Garcia-Molina et al.

## 18.1.2 Serial Schedules

A schedule is *serial* if its actions consist of all the actions of one transaction, then all the actions of another transaction, and so on. No mixing of the actions

- Assume there are three
  - concurrently executing transactions allowed.
  - T1, T2 and T3
- The transaction manager
  - Enables concurrent execution
  - But schedules individual operations
  - To ensure that the final DB state
  - Is *equivalent* to one of the following schedules
    - T1, T2, T3
    - T1, T3, T2
    - T2, T1, T3
    - T2, T3, T1
    - T3, T1, T2
    - T3, T2, T1

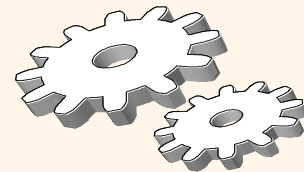
Concurrent execution was *serializable*.

| $T_1$      | $T_2$      | A   | B   |
|------------|------------|-----|-----|
|            |            | 25  | 25  |
| READ(A,t)  |            |     |     |
| t := t+100 |            |     |     |
| WRITE(A,t) |            | 125 |     |
| READ(B,t)  |            |     |     |
| t := t+100 |            |     |     |
| WRITE(B,t) |            |     | 125 |
|            | READ(A,s)  |     |     |
|            | s := s*2   |     |     |
|            | WRITE(A,s) | 250 |     |
|            | READ(B,s)  |     |     |
|            | s := s*2   |     |     |
|            | WRITE(B,s) |     | 250 |

Figure 18.3: Serial schedule in which  $T_1$  precedes  $T_2$

# Serializability ([en.wikipedia.org/wiki/Serializability](https://en.wikipedia.org/wiki/Serializability))

- **Serializability** is used to keep the data in the data item in a consistent state. Serializability is a property of a transaction [schedule](#) (history). It relates to the [isolation](#) property of a [database transaction](#).
- **Serializability** of a schedule means equivalence (in the outcome, the database state, data values) to a *serial schedule* (i.e., sequential with no transaction overlap in time) with the same transactions. It is the major criterion for the correctness of concurrent transactions' schedule, and thus supported in all general purpose database systems.
- **The rationale behind serializability** is the following:
  - If each transaction is correct by itself, i.e., meets certain integrity conditions,
  - then a schedule that comprises any *serial* execution of these transactions is correct (its transactions still meet their conditions):
    - "Serial" means that transactions do not overlap in time and cannot interfere with each other, i.e., complete *isolation* between each other exists.
    - Any order of the transactions is legitimate, (...)
    - As a result, a schedule that comprises any execution (not necessarily serial) that is equivalent (in its outcome) to any serial execution of these transactions, is correct.



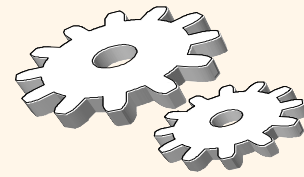
# Lock-Based Concurrency Control

## ❖ Strict Two-phase Locking (Strict 2PL) Protocol:

- Each Xact must obtain a **S (shared) lock** on object before reading, and an **X (exclusive) lock** on object before writing.
- All locks held by a transaction are released when the transaction completes
  - **(Non-strict) 2PL Variant:** Release locks anytime, but cannot acquire locks after releasing any lock.
- If an Xact holds an X lock on an object, no other Xact can get a lock (S or X) on that object.

## ❖ Strict 2PL allows only serializable schedules.

- Additionally, it simplifies transaction aborts
- **(Non-strict) 2PL** also allows only serializable schedules, but involves more complex abort processing



# Aborting a Transaction

- ❖ If a transaction  $T_i$  is aborted, all its actions have to be undone. Not only that, if  $T_j$  reads an object last written by  $T_i$ ,  $T_j$  must be aborted as well!
- ❖ Most systems avoid such *cascading aborts* by releasing a transaction's locks only at commit time.
  - If  $T_i$  writes an object,  $T_j$  can read this only after  $T_i$  commits.
- ❖ In order to *undo* the actions of an aborted transaction, the DBMS maintains a *log* in which every write is recorded. This mechanism is also used to recover from system crashes: all active Xacts at the time of the crash are aborted when the system comes back up.

# MySQL (Locking) Isolation

## 13.3.6 SET TRANSACTION Syntax

```
1  SET [GLOBAL | SESSION] TRANSACTION
2      transaction_characteristic [, transaction_characteristic] ...
3
4  transaction_characteristic:
5      ISOLATION LEVEL level
6      | READ WRITE
7      | READ ONLY
8
9  level:
10     REPEATABLE READ
11     | READ COMMITTED
12     | READ UNCOMMITTED
13     | SERIALIZABLE
```

### Scope of Transaction Characteristics

You can set transaction characteristics globally, for the current session, or for the next transaction:

- With the `GLOBAL` keyword, the statement applies globally for all subsequent sessions. Existing sessions are unaffected.
- With the `SESSION` keyword, the statement applies to all subsequent transactions performed within the current session.
- Without any `SESSION` or `GLOBAL` keyword, the statement applies to the next (not started) transaction performed within the current session. Subsequent transactions revert to using the `SESSION` isolation level.

# Isolation Levels

([https://en.wikipedia.org/wiki/Isolation\\_\(database\\_systems\)](https://en.wikipedia.org/wiki/Isolation_(database_systems)))

Not all transaction use cases require 2PL and serializable execution. Databases support a set of levels.

- **Serializable**

- With a lock-based [concurrency control](#) DBMS implementation, [serializability](#) requires read and write locks (acquired on selected data) to be released at the end of the transaction. Also *range-locks* must be acquired when a [SELECT](#) query uses a ranged *WHERE* clause, especially to avoid the [phantom reads](#) phenomenon.
- *The execution of concurrent SQL-transactions at isolation level SERIALIZABLE is guaranteed to be serializable. A serializable execution is defined to be an execution of the operations of concurrently executing SQL-transactions that produces the same effect as some serial execution of those same SQL-transactions. A serial execution is one in which each SQL-transaction executes to completion before the next SQL-transaction begins.*

- **Repeatable reads**

- In this isolation level, a lock-based [concurrency control](#) DBMS implementation keeps read and write locks (acquired on selected data) until the end of the transaction. However, *range-locks* are not managed, so [phantom reads](#) can occur.
- Write skew is possible at this isolation level, a phenomenon where two writes are allowed to the same column(s) in a table by two different writers (who have previously read the columns they are updating), resulting in the column having data that is a mix of the two transactions. [\[3\]\[4\]](#)

- **Read committed**

- In this isolation level, a lock-based [concurrency control](#) DBMS implementation keeps write locks (acquired on selected data) until the end of the transaction, but read locks are released as soon as the [SELECT](#) operation is performed (so the [non-repeatable reads phenomenon](#) can occur in this isolation level). As in the previous level, *range-locks* are not managed.
- Putting it in simpler words, read committed is an isolation level that guarantees that any data read is committed at the moment it is read. It simply restricts the reader from seeing any intermediate, uncommitted, 'dirty' read. It makes no promise whatsoever that if the transaction re-issues the read, it will find the same data; data is free to change after it is read.

- **Read uncommitted**

- This is the *lowest* isolation level. In this level, [dirty reads](#) are allowed, so one transaction may see *not-yet-committed* changes made by other transactions

# In Databases, Cursors Define *Isolation*

- We have talked about ACID transactions

| Isolation level  | Dirty reads | Non-repeatable reads | Phantoms  |
|------------------|-------------|----------------------|-----------|
| Read Uncommitted | may occur   | may occur            | may occur |
| Read Committed   | -           | may occur            | may occur |
| Repeatable Read  | -           | -                    | may occur |
| Serializable     | -           | -                    | -         |

```
InitialContext ctx = new InitialContext();
DataSource ds = (DataSource)
ctx.lookup("jdbc/MyBase");
Connection con = ds.getConnection();
DatabaseMetaData dbmd = con.getMetaData();
if (dbmd.supportsTransactionIsolationLevel(TRANSACTION_SERIALIZABLE)
{ Connection.setTransactionIsolation(TRANSACTION_SERIALIZABLE); }
```

- Isolation
  - Determines what happens when two or more threads are manipulating the data at the same time.
  - And is defined relative to where cursors are and what they have touched.
  - Because the cursor movement determines *what you are reading or have read*.
- *But, ... Cursors are client conversation state and cannot be used in REST.*

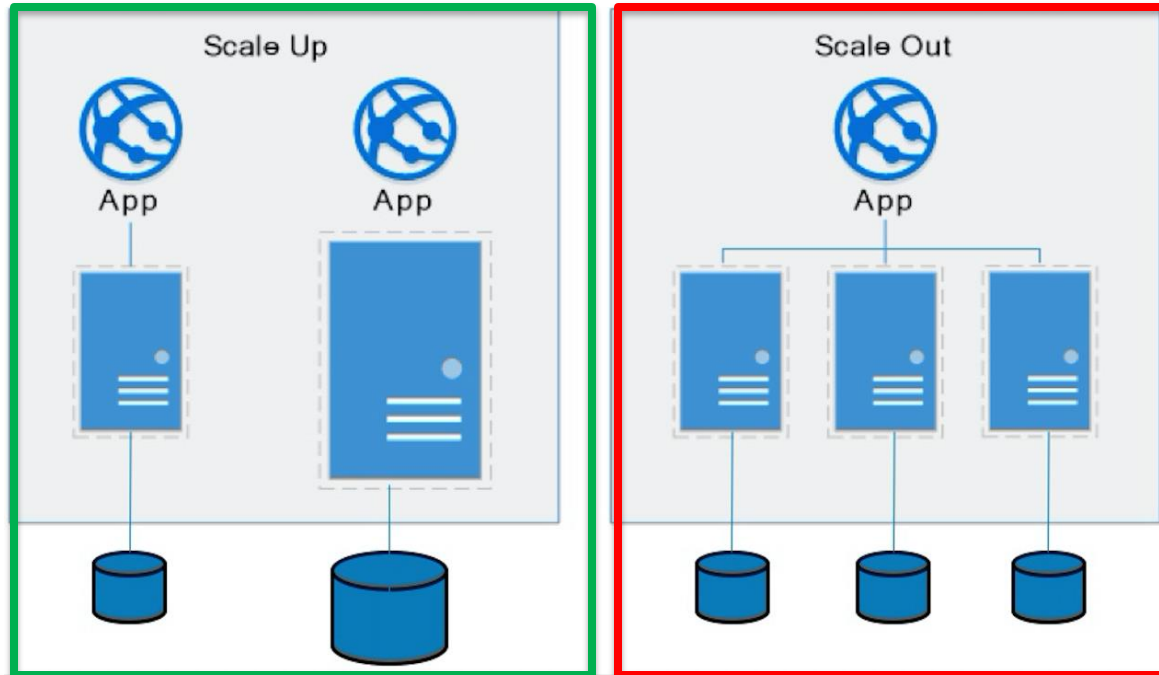
# *Scalability* *Availability*

# Approaches to Scalability

**Scalability** is the property of a system to handle a growing amount of work by adding resources to the system.

Replace system with a bigger machine,  
e.g. more memory, CPU, ... ..

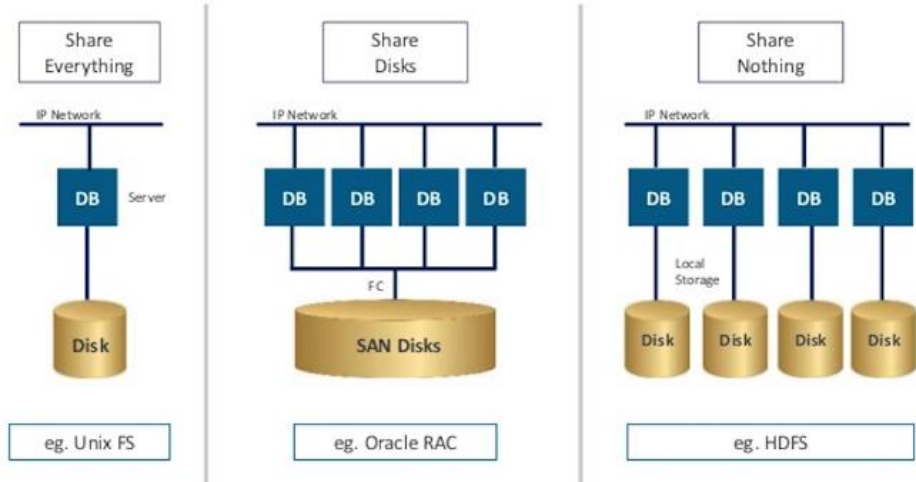
Add another system.



- Scale-up:
  - Less incremental.
  - More disruptive.
  - More expensive for extremely large systems.
  - Does not improve availability
- Scale-out:
  - Incremental cost.
  - Data replication enables availability.
  - Does not work well for functions like JOIN, referential integrity, ... ..

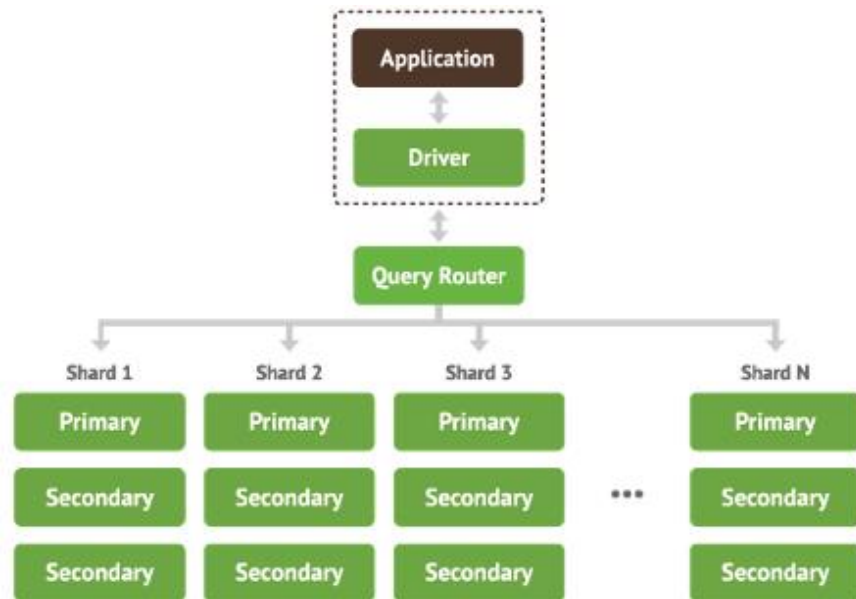
# Disk Architecture for Scale-Out

- Share disks:
  - Is basically scale-up for data/disks. You can use NAS, SAN and RAID.
  - Isolation/Integrity requires distributed locking to control access from multiple database servers.
- Share nothing:
  - Is basically scale-out for disks.
  - Data is partitioned into *shards* based on a function  $f()$  applied to a key.
  - Can improve availability, at the code consistency, with data replication.
  - There is a router that sends requests to the proper shard based on the function.

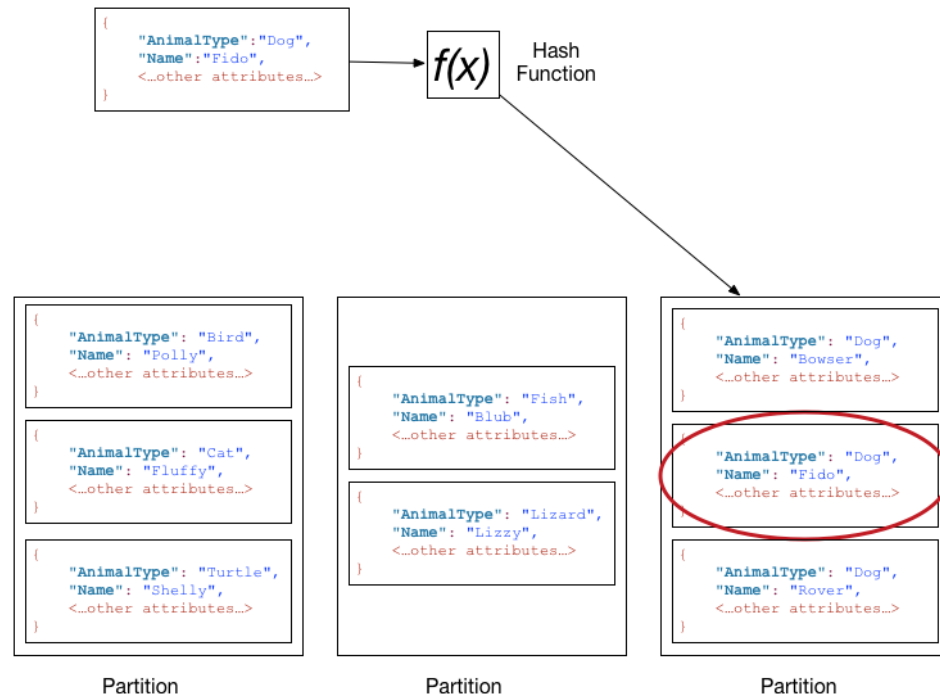


# Shared Nothing, Scale-Out

## MongoDB Sharding



## DynamoDB Partitioning



# Data Enablement Decision Support

# Overview



# Overview

- **Data analytics:** the processing of data to infer patterns, correlations, or models for prediction
- Primarily used to make business decisions
  - Per individual customer
    - E.g., what product to suggest for purchase
  - Across all customers
    - E.g., what products to manufacture/stock, in what quantity
- Critical for businesses today



## Overview (Cont.)

- Common steps in data analytics
  - Gather data from multiple sources into one location
    - Data warehouses also integrated data into common schema
    - Data often needs to be **extracted** from source formats, **transformed** to common schema, and **loaded** into the data warehouse
      - Can be done as **ETL (extract-transform-load)**, or **ELT (extract-load-transform)**
  - Generate aggregates and reports summarizing data
    - Dashboards showing graphical charts/reports
    - **Online analytical processing (OLAP) systems** allow interactive querying
    - Statistical analysis using tools such as R/SAS/SPSS
      - Including extensions for parallel processing of big data
  - Build **predictive models** and use the models for decision making



## Overview (Cont.)

- Predictive models are widely used today
  - E.g., use customer profile features (e.g. income, age, gender, education, employment) and past history of a customer to predict likelihood of default on loan
    - and use prediction to make loan decision
  - E.g., use past history of sales (by season) to predict future sales
    - And use it to decide what/how much to produce/stock
    - And to target customers
- Other examples of business decisions:
  - What items to stock?
  - What insurance premium to change?
  - To whom to send advertisements?



## Overview (Cont.)

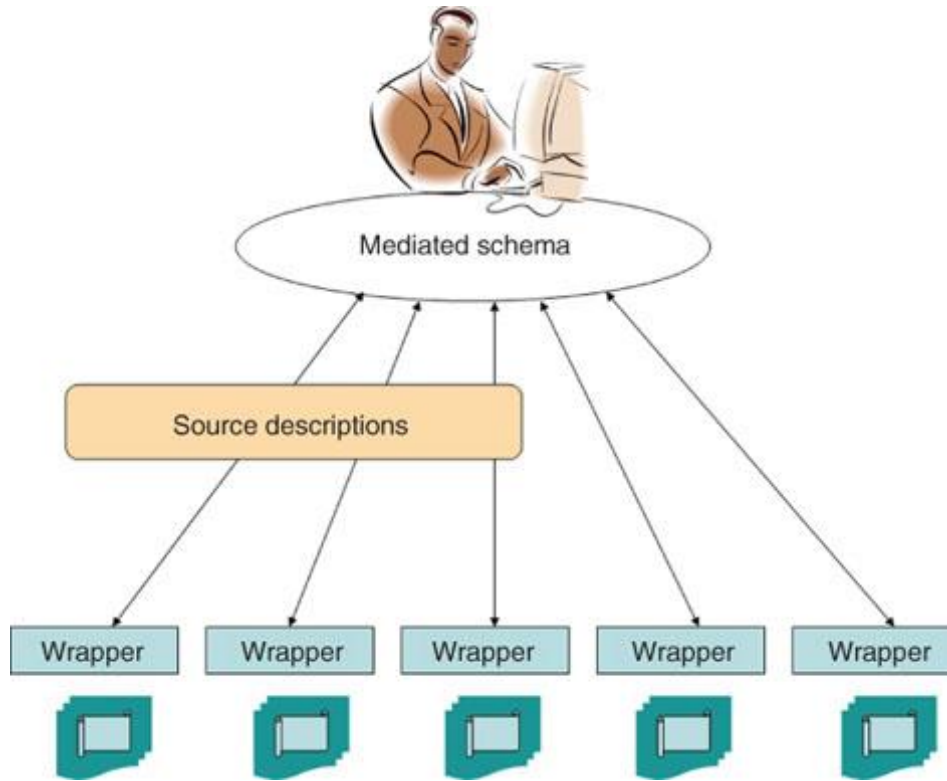
- **Machine learning** techniques are key to finding patterns in data and making predictions
- **Data mining** extends techniques developed by machine-learning communities to run them on very large datasets
- The term **business intelligence (BI)** is synonym for data analytics
- The term **decision support** focuses on reporting and aggregation

# Enterprise Information Integration

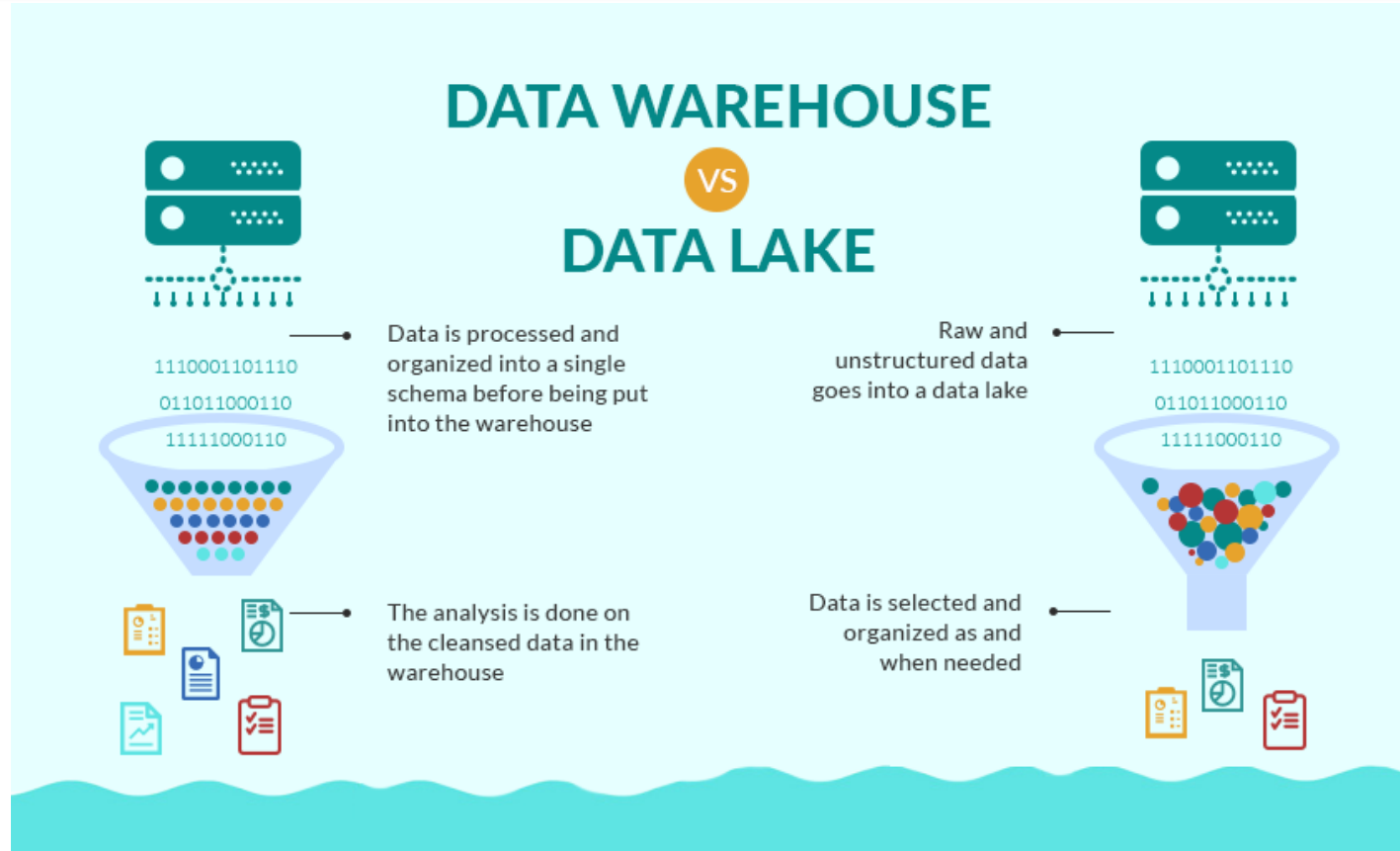
## ETL

### Data Warehouse, Data Lake

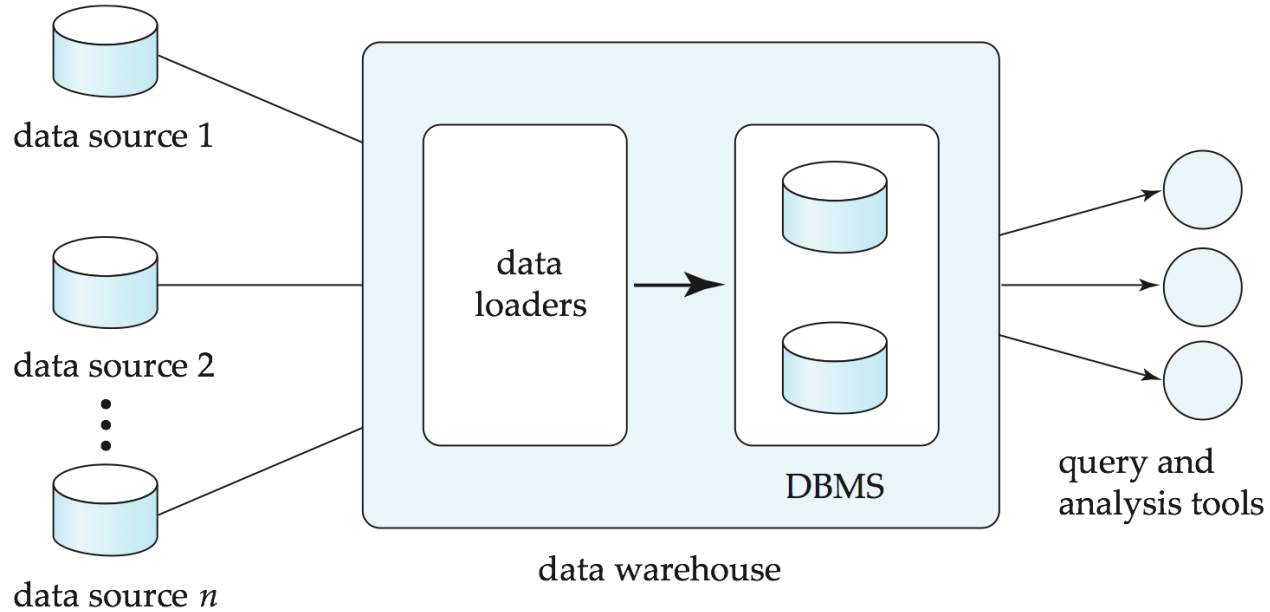
# Enterprise Information Integration



# Data Warehouse and Data Lake



# Data Warehousing

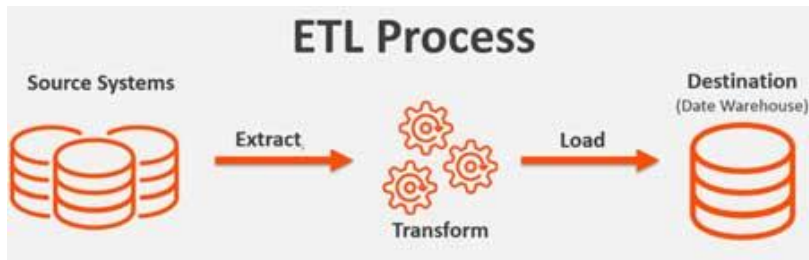


# Overview (Cont.)

- Common steps in data analytics
  - Gather data from multiple sources into one location
    - Data warehouses also integrated data into common schema
    - Data often needs to be **extracted** from source formats, **transformed** to common schema, and **loaded** into the data warehouse
      - Can be done as **ETL (extract-transform-load)**, or **ELT (extract-load-transform)**
  - Generate aggregates and reports summarizing data
    - Dashboards showing graphical charts/reports
    - **Online analytical processing (OLAP) systems** allow interactive querying
    - Statistical analysis using tools such as R/SAS/SPSS
      - Including extensions for parallel processing of big data
  - Build **predictive models** and use the models for decision making

# ETL Concepts

<https://databricks.com/glossary/extract-transform-load>



## Extract

The first step of this process is extracting data from the target sources that could include an ERP, CRM, Streaming sources, and other enterprise systems as well as data from third-party sources. There are different ways to perform the extraction: **Three**

### Data Extraction methods:

1. Partial Extraction – The easiest way to obtain the data is if the source system notifies you when a record has been changed
2. Partial Extraction- with update notification – Not all systems can provide a notification in case an update has taken place; however, they can point those records that have been changed and provide an extract of such records.
3. Full extract – There are certain systems that cannot identify which data has been changed at all. In this case, a full extract is the only possibility to extract the data out of the system. This method requires having a copy of the last extract in the same format so you can identify the changes that have been made.

## Transform

Next, the transform function converts the raw data that has been extracted from the source server. As it cannot be used in its original form in this stage it gets cleansed, mapped and transformed, often to a specific data schema, so it will meet operational needs. This process entails several transformation types that ensure the quality and integrity of data; below are the most common as well as advanced transformation types that prepare data for analysis:

- Basic transformations:
- Cleaning
- Format revision
- Data threshold validation checks
- Restructuring
- Deduplication
- Advanced transformations:
- Filtering
- Merging
- Splitting
- Derivation
- Summarization
- Integration
- Aggregation
- Complex data validation

## Load

Finally, the load function is the process of writing converted data from a staging area to a target database, which may or may not have previously existed. Depending on the requirements of the application, this process may be either quite simple or intricate.

# *OLAP*



# Data Analysis and OLAP

- **Online Analytical Processing (OLAP)**

- Interactive analysis of data, allowing data to be summarized and viewed in different ways in an online fashion (with negligible delay)

- We use the following relation to illustrate OLAP concepts

- *sales (item\_name, color, clothes\_size, quantity)*

This is a simplified version of the *sales* fact table joined with the dimension tables, and many attributes removed (and some renamed)



# Example sales relation

| <i>item_name</i> | <i>color</i> | <i>clothes_size</i> | <i>quantity</i> |
|------------------|--------------|---------------------|-----------------|
| dress            | dark         | small               | 2               |
| dress            | dark         | medium              | 6               |
| dress            | dark         | large               | 12              |
| dress            | pastel       | small               | 4               |
| dress            | pastel       | medium              | 3               |
| dress            | pastel       | large               | 3               |
| dress            | white        | small               | 2               |
| dress            | white        | medium              | 3               |
| dress            | white        | large               | 0               |
| pants            | dark         | small               | 14              |
| pants            | dark         | medium              | 6               |
| pants            | dark         | large               | 0               |
| pants            | pastel       | small               | 1               |
| pants            | pastel       | medium              | 0               |
| pants            | pastel       | large               | 1               |
| pants            | white        | small               | 3               |
| pants            | white        | medium              | 0               |
| pants            | white        | large               | 2               |
| shirt            | dark         | small               | 2               |
| shirt            | dark         | medium              | 6               |
| shirt            | dark         | large               | 6               |
| shirt            | pastel       | small               | 4               |
| shirt            | pastel       | medium              | 1               |
| shirt            | pastel       | large               | 2               |
| shirt            | white        | small               | 17              |
| shirt            | white        | medium              | 1               |
| shirt            | white        | large               | 10              |
| skirt            | dark         | small               | 2               |
| skirt            | dark         | medium              | 5               |
| ...              | ...          | ...                 | ...             |
| ...              | ...          | ...                 | ...             |



# Cross Tabulation of sales by *item\_name* and *color*

*clothes\_size* **all**

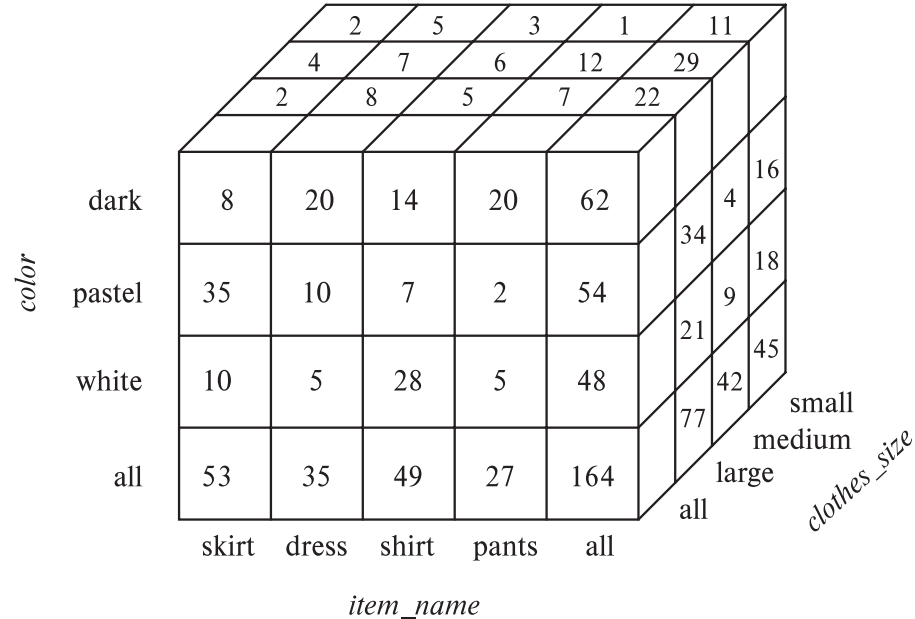
|                  |       | <i>color</i> |        |       |       |
|------------------|-------|--------------|--------|-------|-------|
|                  |       | dark         | pastel | white | total |
| <i>item_name</i> | skirt | 8            | 35     | 10    | 53    |
|                  | dress | 20           | 10     | 5     | 35    |
|                  | shirt | 14           | 7      | 28    | 49    |
|                  | pants | 20           | 2      | 5     | 27    |
|                  | total | 62           | 54     | 48    | 164   |

- The table above is an example of a **cross-tabulation (cross-tab)**, also referred to as a **pivot-table**.
  - Values for one of the dimension attributes form the row headers
  - Values for another dimension attribute form the column headers
  - Other dimension attributes are listed on top
  - Values in individual cells are (aggregates of) the values of the dimension attributes that specify the cell.



# Data Cube

- A **data cube** is a multidimensional generalization of a cross-tab
- Can have n dimensions; we show 3 below
- Cross-tabs can be used as views on a data cube





# Online Analytical Processing Operations

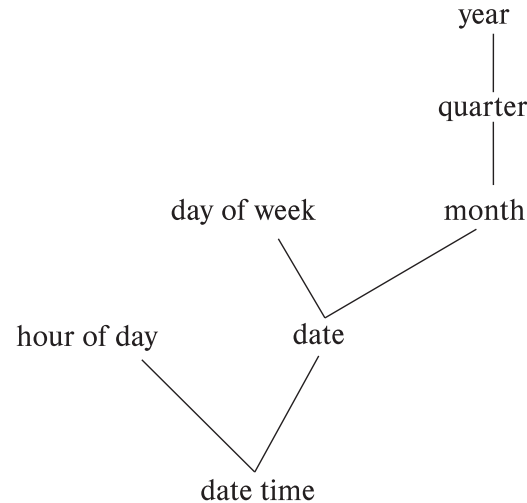
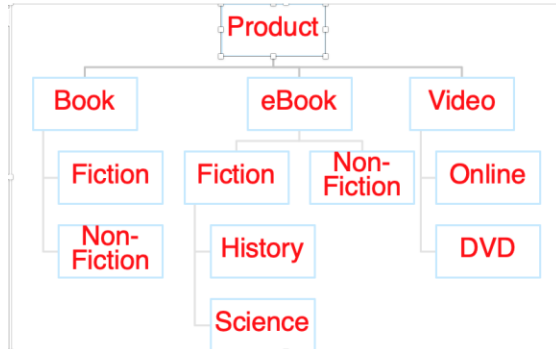
- **Pivoting:** changing the dimensions used in a cross-tab
  - E.g., moving colors to column names
- **Slicing:** creating a cross-tab for fixed values only
  - E.g., fixing color to white and size to small
  - Sometimes called **dicing**, particularly when values for multiple dimensions are fixed.
- **Rollup:** moving from finer-granularity data to a coarser granularity
  - E.g., aggregating away an attribute
  - E.g., moving from aggregates by day to aggregates by month or year
- **Drill down:** The opposite operation - that of moving from coarser-granularity data to finer-granularity data



# Hierarchies on Dimensions

- **Hierarchy** on dimension attributes: lets dimensions be viewed at different levels of detail
- E.g., the dimension *datetime* can be used to aggregate by hour of day, date, day of week, month, quarter or year

Another dimension could be ...  
Product category.

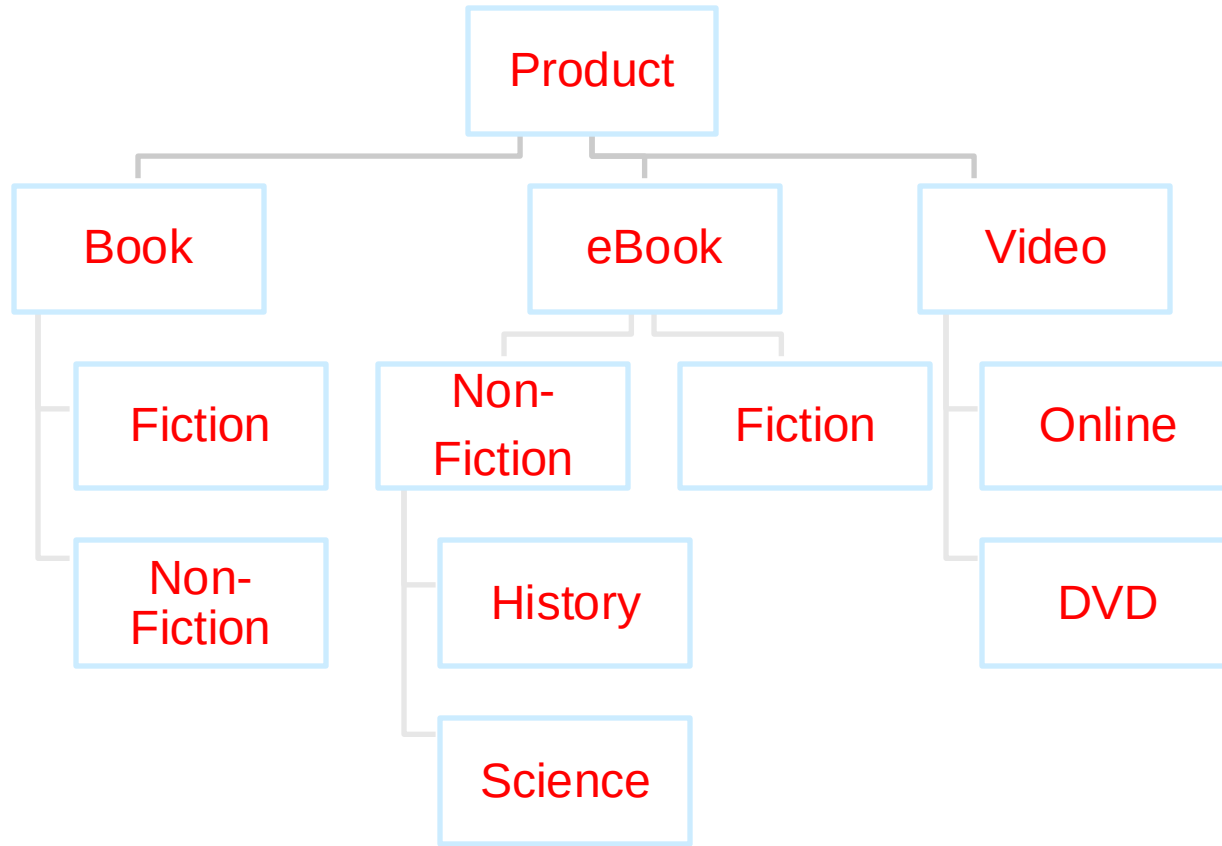


(a) time hierarchy



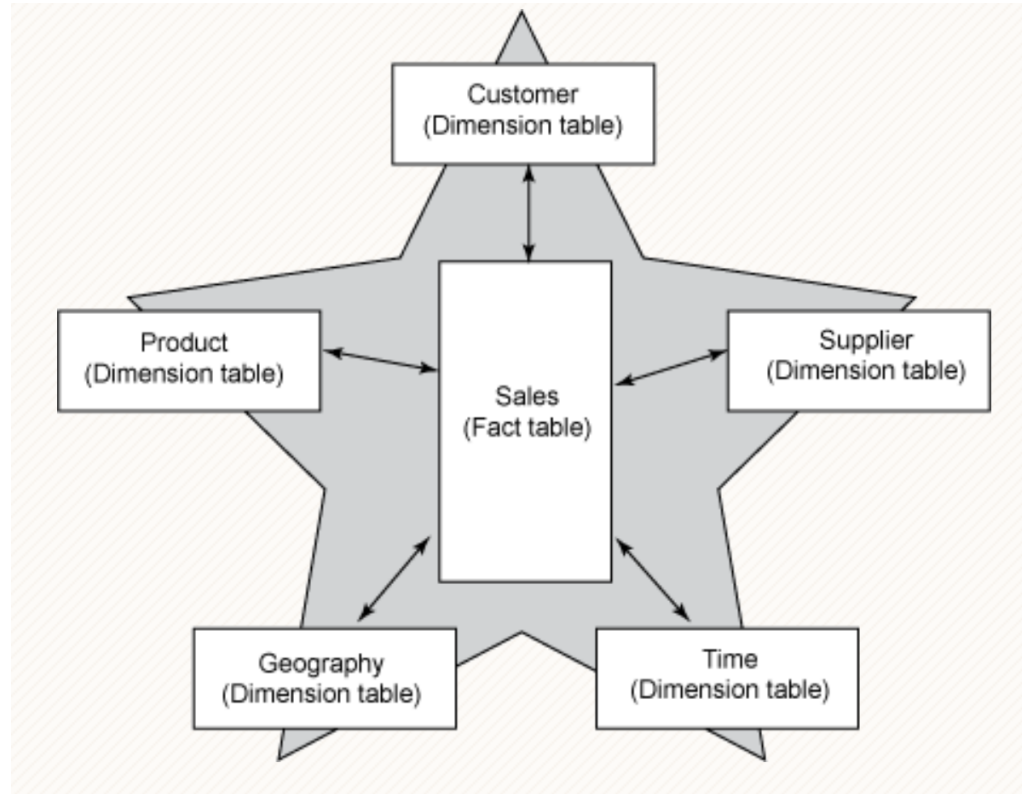
(b) location hierarchy

# Another Dimension Example – Product Categories





# Facts and Dimensions

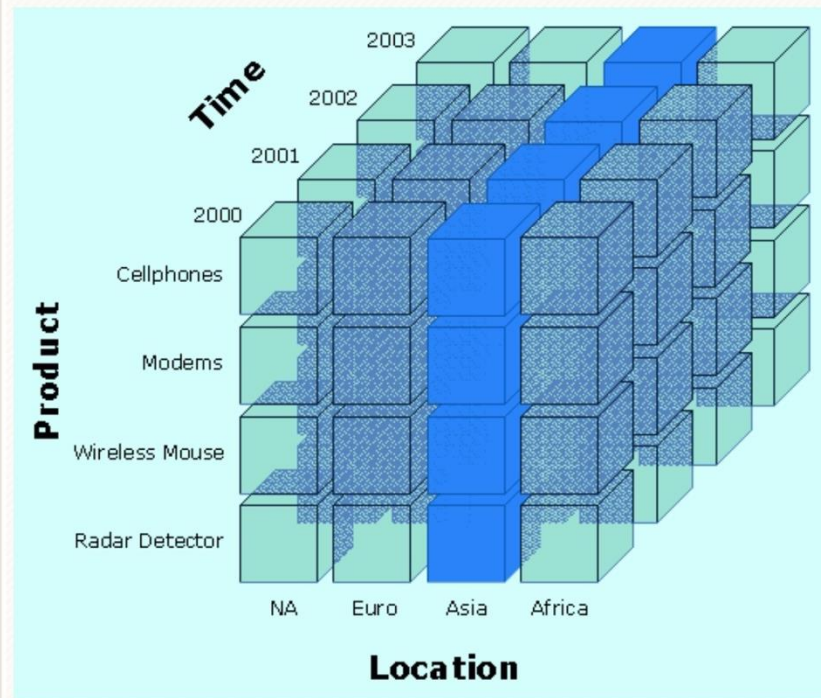




# Slice

## *Slice:*

A slice is a subset of a multi-dimensional array corresponding to a single value for one or more members of the dimensions not in the subset.

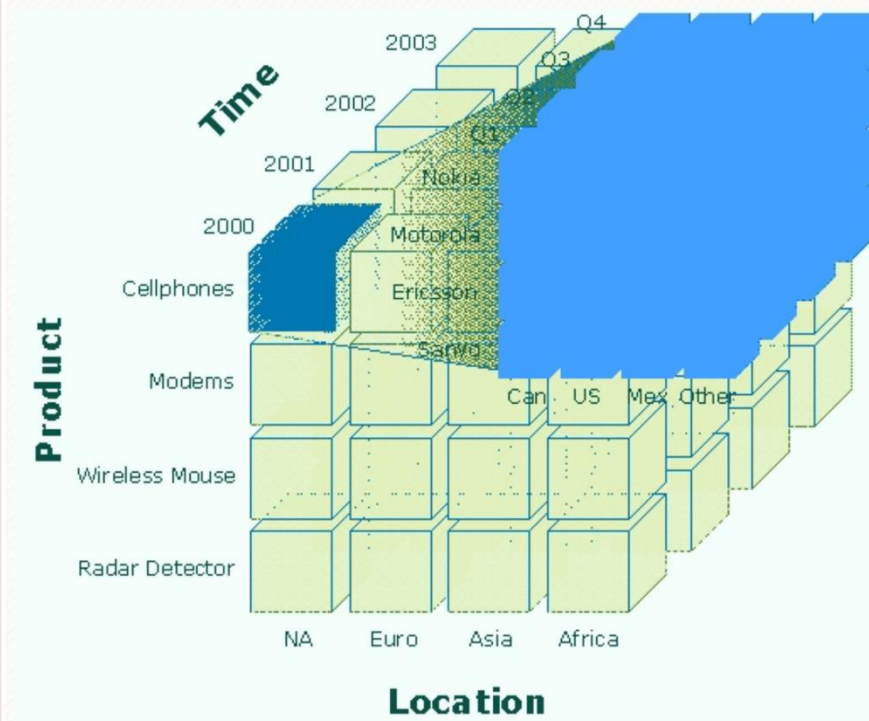




# Dice

## *Dice:*

The dice operation is a slice on more than two dimensions of a data cube (or more than two consecutive slices).

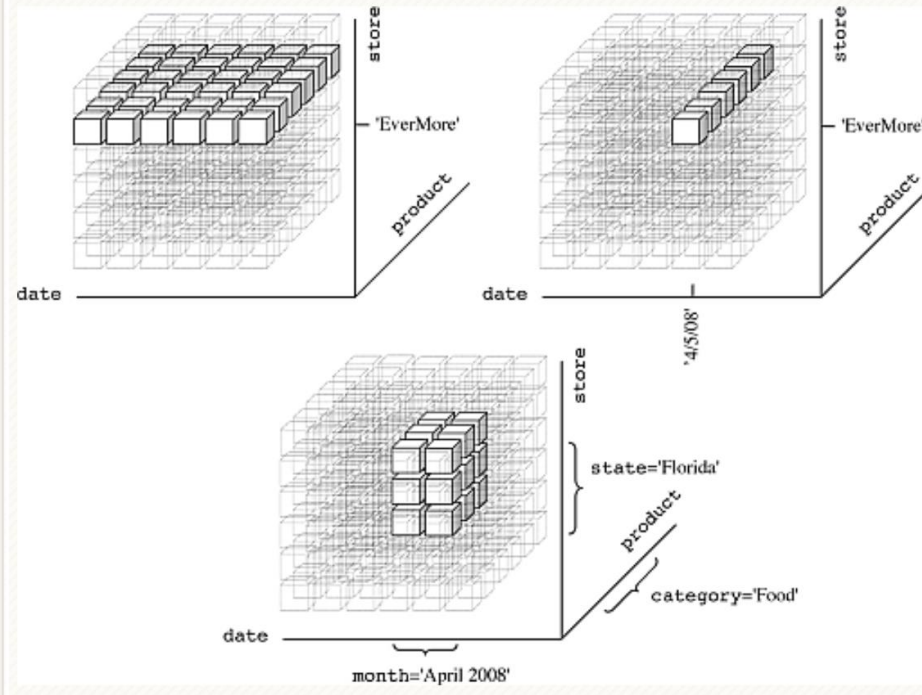




# Drilling

## *Drill Down/Up:*

Drilling down or up is a specific analytical technique whereby the user navigates among levels of data ranging from the most summarized (up) to the most detailed (down).

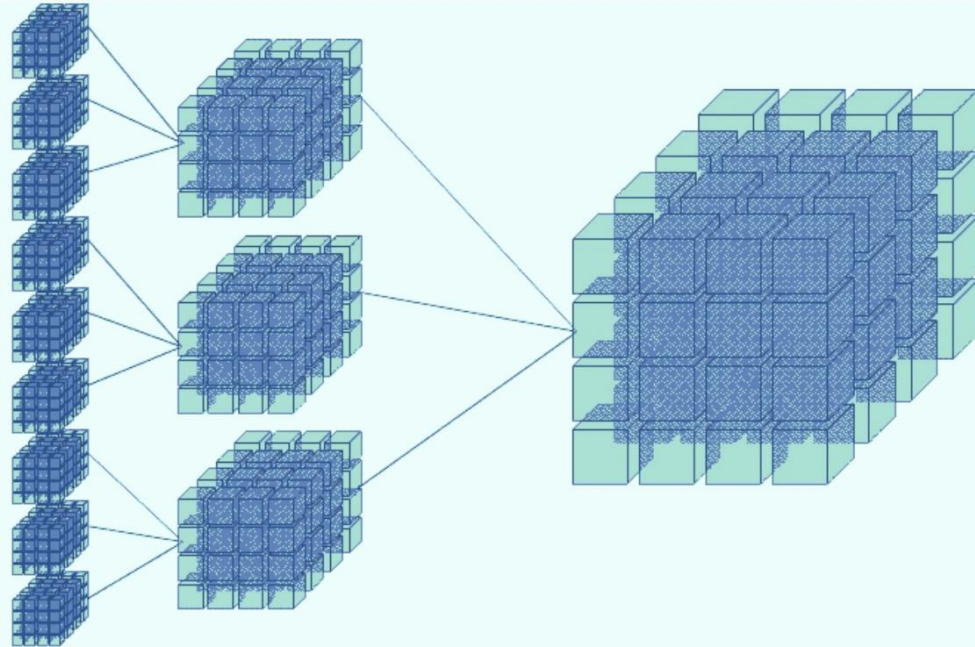




# Roll-up

## *Roll-up:*

(Aggregate, Consolidate) A roll-up involves computing all of the data relationships for one or more dimensions. To do this, a computational relationship or formula might be defined.

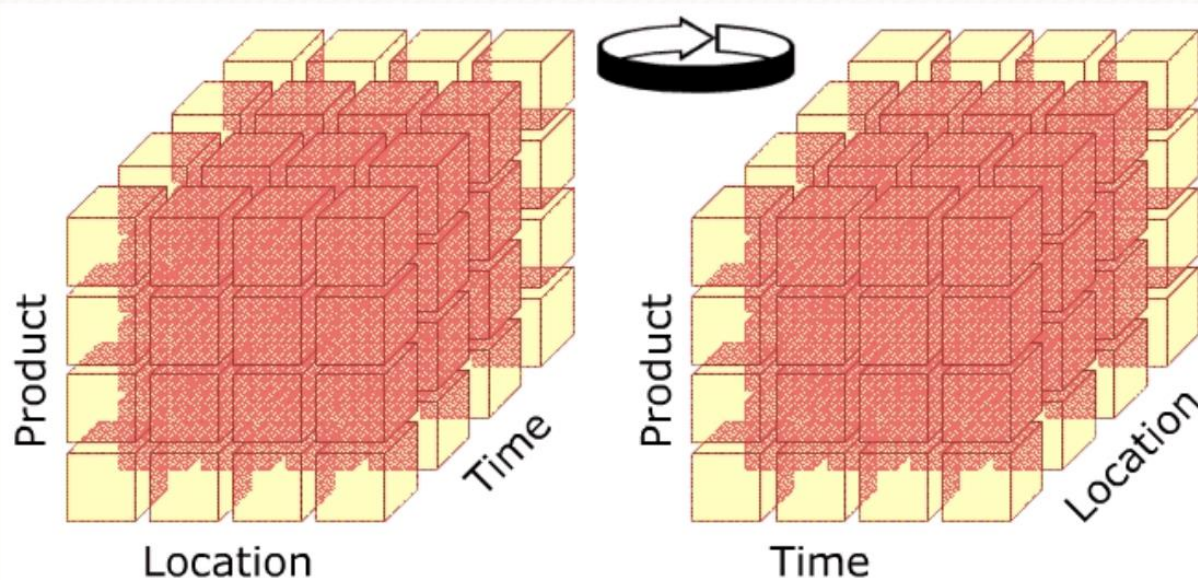




# Pivot

## *Pivot:*

This operation is also called rotate operation. It rotates the data in order to provide an alternative presentation of data – the report or page display takes a different dimensional orientation.





# Cross Tabulation With Hierarchy

- Cross-tabs can be easily extended to deal with hierarchies
- Can drill down or roll up on a hierarchy
- E.g. hierarchy: *item\_name* → *category*

*clothes\_size:* **all**

| <i>category</i> |          | <i>item_name</i> |        | <i>color</i> |       |     |
|-----------------|----------|------------------|--------|--------------|-------|-----|
|                 |          | dark             | pastel | white        | total |     |
| womenswear      | skirt    | 8                | 8      | 10           | 53    | 88  |
|                 | dress    | 20               | 20     | 5            | 35    |     |
|                 | subtotal | 28               | 28     | 15           |       |     |
| menswear        | pants    | 14               | 14     | 28           | 49    | 76  |
|                 | shirt    | 20               | 20     | 5            | 27    |     |
|                 | subtotal | 34               | 34     | 33           |       |     |
| total           |          | 62               | 62     | 48           |       | 164 |



# Relational Representation of Cross-tabs

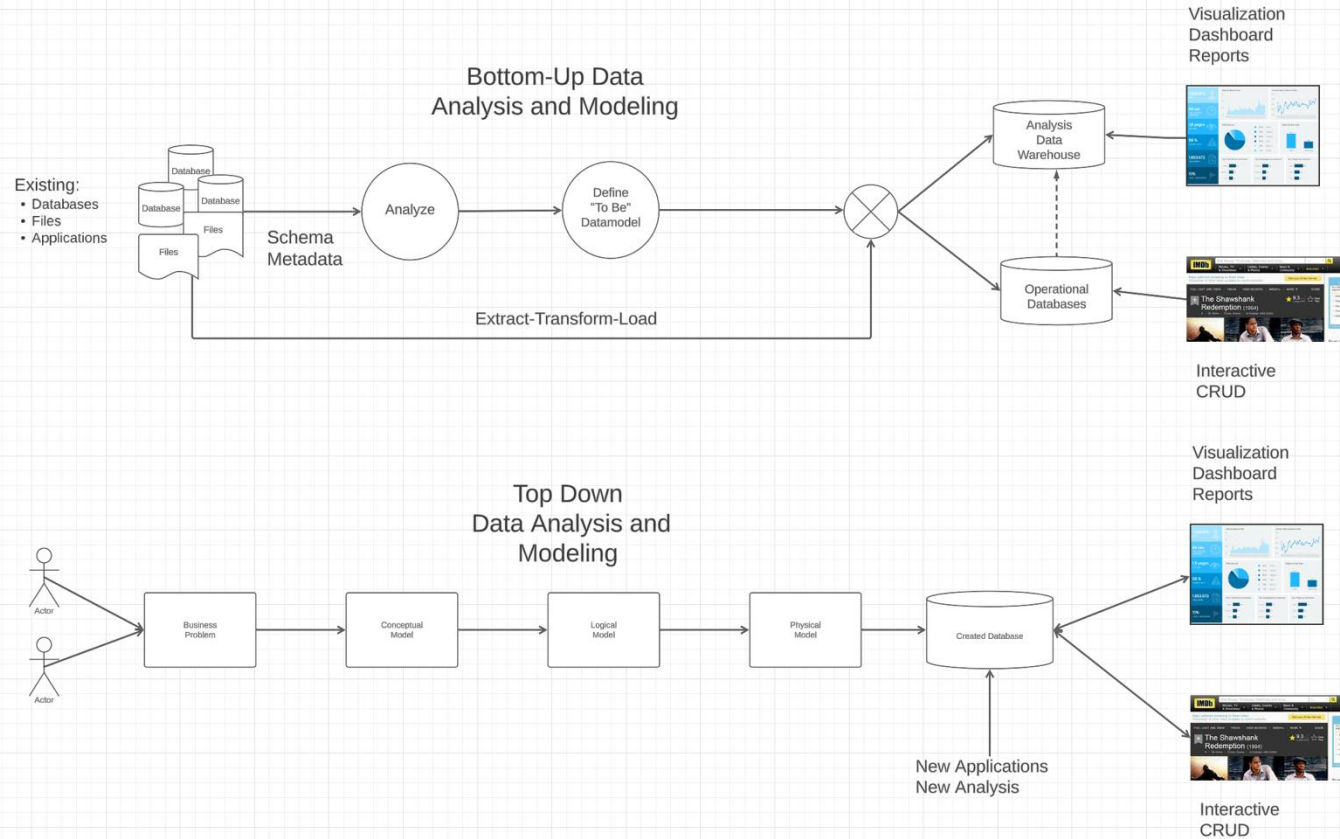
- Cross-tabs can be represented as relations
- We use the value **all** to represent aggregates.
- The SQL standard actually uses *null* values in place of **all**
  - Works with any data type
  - But can cause confusion with regular null values.

| <i>item_name</i> | <i>color</i> | <i>clothes_size</i> | <i>quantity</i> |
|------------------|--------------|---------------------|-----------------|
| skirt            | dark         | <b>all</b>          | 8               |
| skirt            | pastel       | <b>all</b>          | 35              |
| skirt            | white        | <b>all</b>          | 10              |
| skirt            | <b>all</b>   | <b>all</b>          | 53              |
| dress            | dark         | <b>all</b>          | 20              |
| dress            | pastel       | <b>all</b>          | 10              |
| dress            | white        | <b>all</b>          | 5               |
| dress            | <b>all</b>   | <b>all</b>          | 35              |
| shirt            | dark         | <b>all</b>          | 14              |
| shirt            | pastel       | <b>all</b>          | 7               |
| shirt            | white        | <b>all</b>          | 28              |
| shirt            | <b>all</b>   | <b>all</b>          | 49              |
| pants            | dark         | <b>all</b>          | 20              |
| pants            | pastel       | <b>all</b>          | 2               |
| pants            | white        | <b>all</b>          | 5               |
| pants            | <b>all</b>   | <b>all</b>          | 27              |
| <b>all</b>       | dark         | <b>all</b>          | 62              |
| <b>all</b>       | pastel       | <b>all</b>          | 54              |
| <b>all</b>       | white        | <b>all</b>          | 48              |
| <b>all</b>       | <b>all</b>   | <b>all</b>          | 164             |

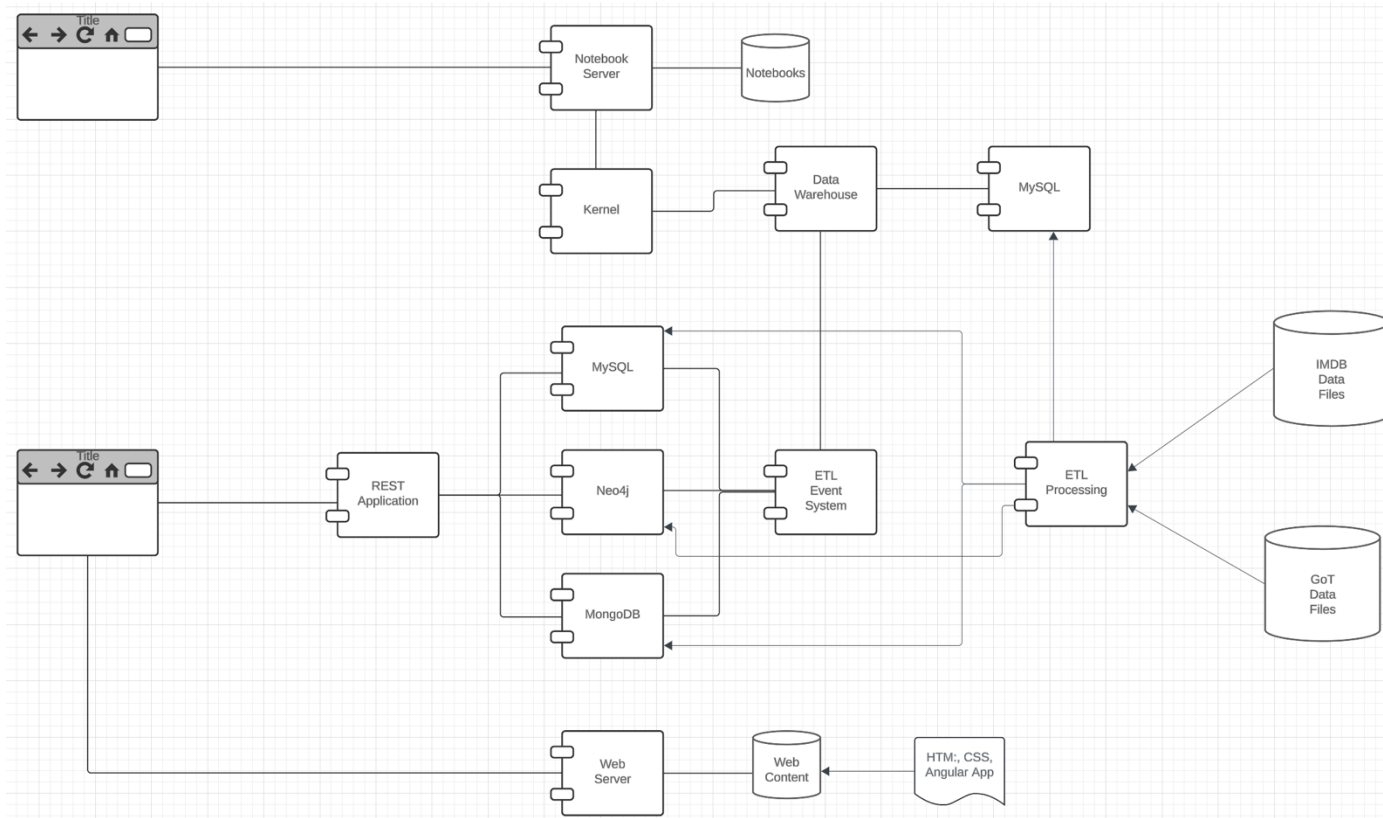
# *HW3 and HW4, “The Project” Discussion*

# *Overview*

# Vision

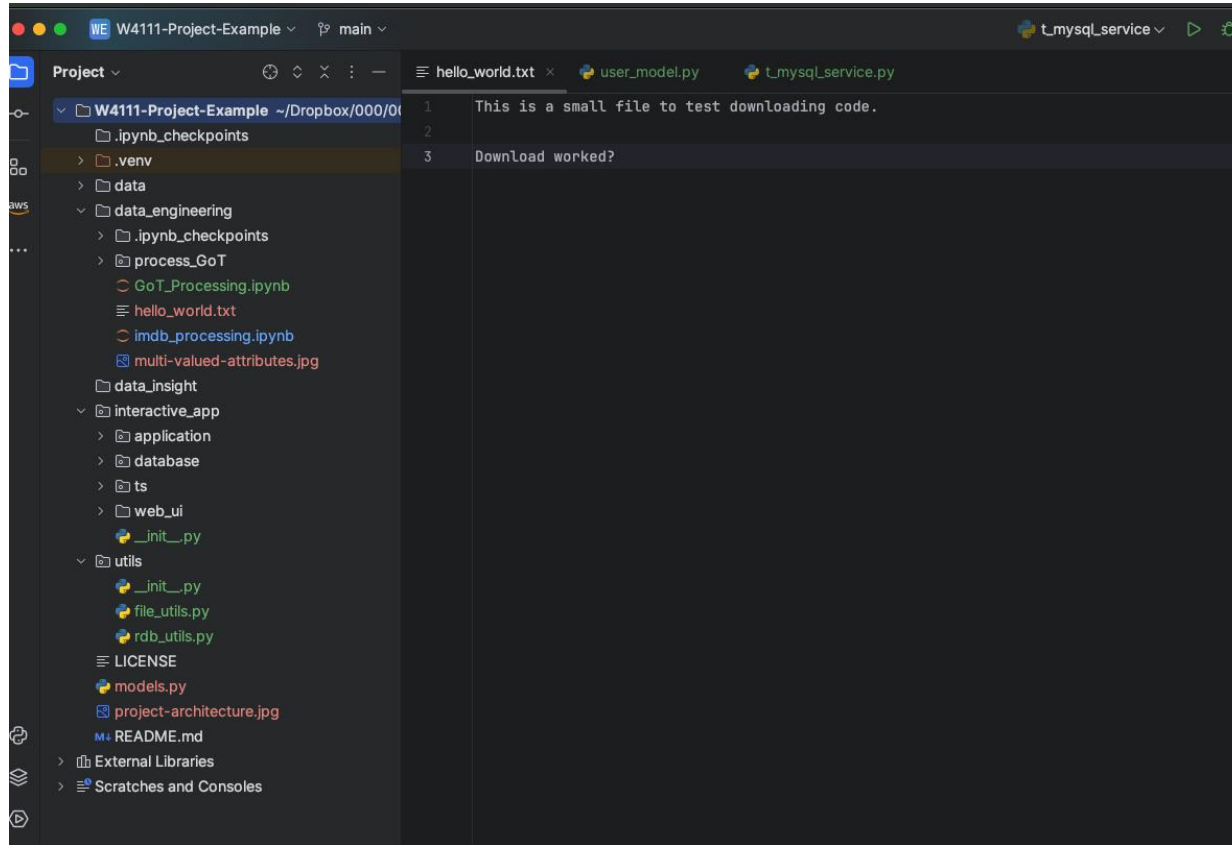


# Vision



- Both programming and non-programming implement a data engineering Jupyter notebook.
- The programming track builds a simple REST application/microservice for transformed data.
- Non-programming implements more complex transformation, and queries for visualization.
- I will provide starter projects with examples.

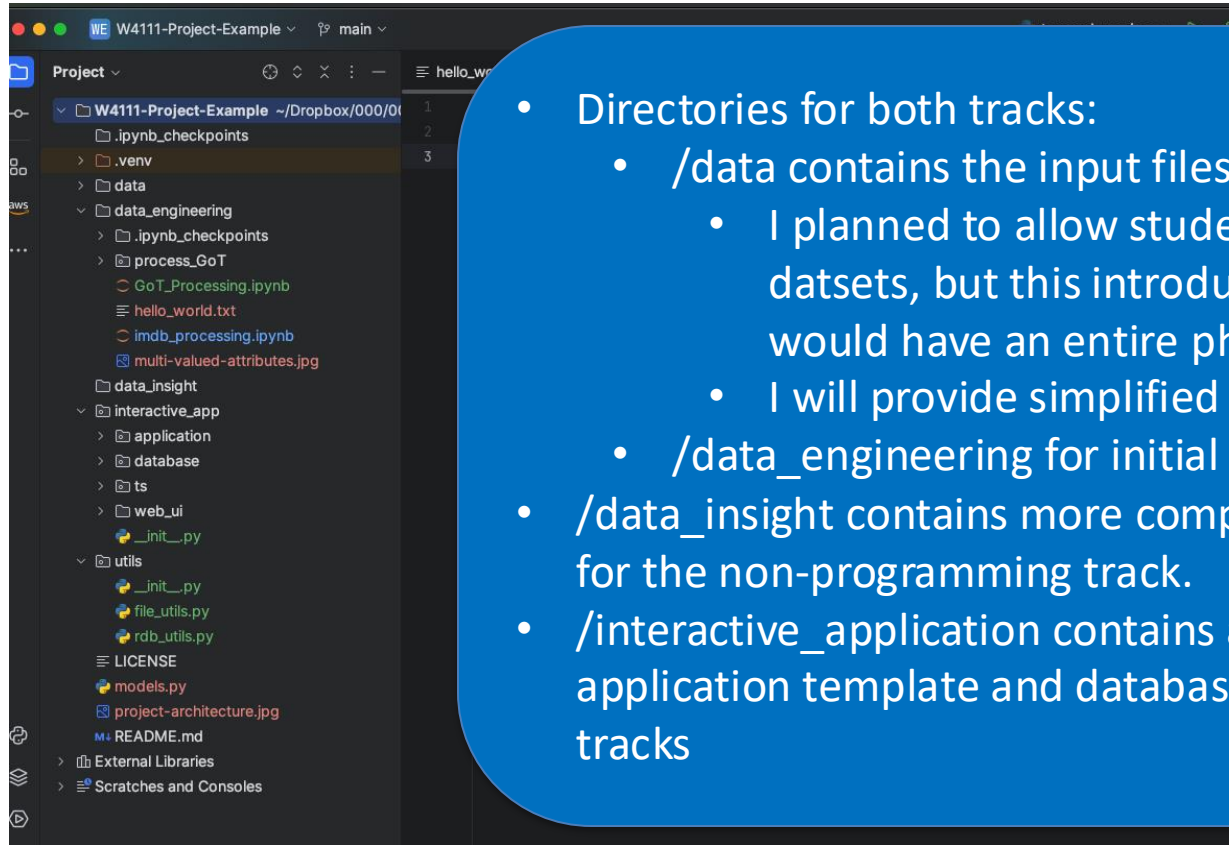
# Hw3, HW4 and Project



The screenshot shows a code editor interface with a project named "W4111-Project-Example" open. The left sidebar displays the project structure, including folders like ".ipynb\_checkpoints", ".venv", "data", "data\_engineering", "data\_insight", "interactive\_app", "utils", and files like "hello\_world.txt", "user\_model.py", "t\_mysql\_service.py", "LICENSE", "models.py", "project-architecture.jpg", and "README.md". The main editor area shows the content of "hello\_world.txt", which contains three lines of text:

```
1 This is a small file to test downloading code.  
2  
3 Download worked?
```

# Hw3, HW4 and Project



- Directories for both tracks:
  - /data contains the input files in CSV and JSON formats
    - I planned to allow students to choose their own datasets, but this introduces too much complexity. We would have an entire phase of “Is this a goo dataset?”
    - I will provide simplified IMDB and GoT data.
  - /data\_engineering for initial ETL
- /data\_insight contains more complex queries and visualization for the non-programming track.
- /interactive\_application contains a simple web UI, REST application template and database schema for the programming tracks

# *REST*

# Data Modeling Concepts and REST

Almost any data model has the same core concepts:

- Types and instances:
  - Entity Type: A definition of a type of thing with properties and relationships.
  - Entity Instance: A specific instantiation of the Entity Type
  - Entity Set Instance: An Entity Type that:
    - Has properties and relationships like any entity, but ...
    - Has at least one *special relationship* – ***contains***.
- Operations, minimally CRUD, that manipulate entity types and instances:
  - Create
  - Retrieve
  - Update
  - Delete
  - Reference/Identify/... ..
  - Host/database/table/pk

## What is REST architecture?

REST stands for REpresentational State Transfer. REST is web standards based architecture and uses HTTP Protocol. It revolves around resource where every component is a resource and a resource is accessed by a common interface using HTTP standard methods. REST was first introduced by Roy Fielding in 2000.

In REST architecture, a REST Server simply provides access to resources and REST client accesses and modifies the resources. Here each resource is identified by URIs/ global IDs. REST uses various representation to represent a resource like text, JSON, XML. JSON is the most popular one.

## HTTP methods

Following four HTTP methods are commonly used in REST based architecture.

- **GET** – Provides a read only access to a resource.
- **POST** – Used to create a new resource.
- **DELETE** – Used to remove a resource.
- **PUT** – Used to update a existing resource or create a new resource.

## Introduction to RESTful web services

A web service is a collection of open protocols and standards used for exchanging data between applications or systems. Software applications written in various programming languages and running on various platforms can use web services to exchange data over computer networks like the Internet in a manner similar to inter-process communication on a single computer. This interoperability (e.g., between Java and Python, or Windows and Linux applications) is due to the use of open standards.

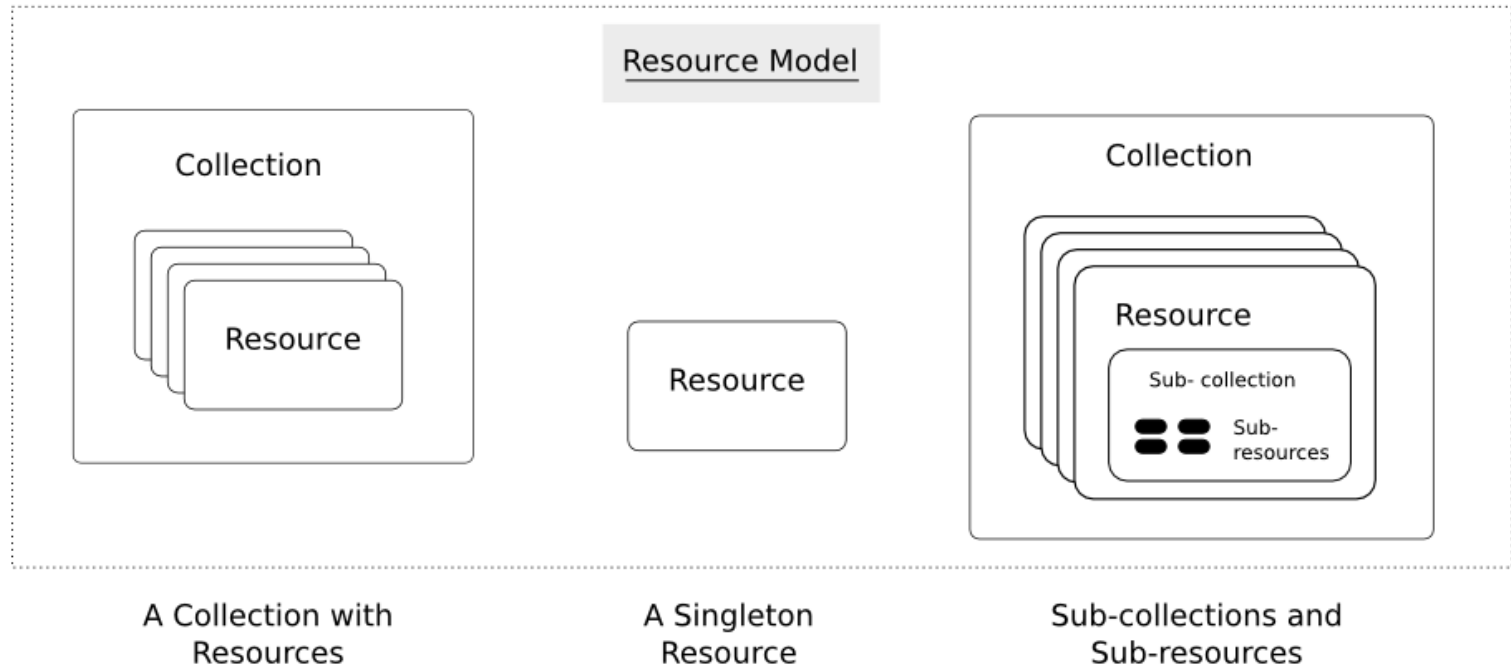
Web services based on REST Architecture are known as RESTful web services. These webservices uses HTTP methods to implement the concept of REST architecture. A RESTful web service usually defines a URI, Uniform Resource Identifier a service, provides resource representation such as JSON and set of HTTP Methods.

## Creating RESTful Webservice

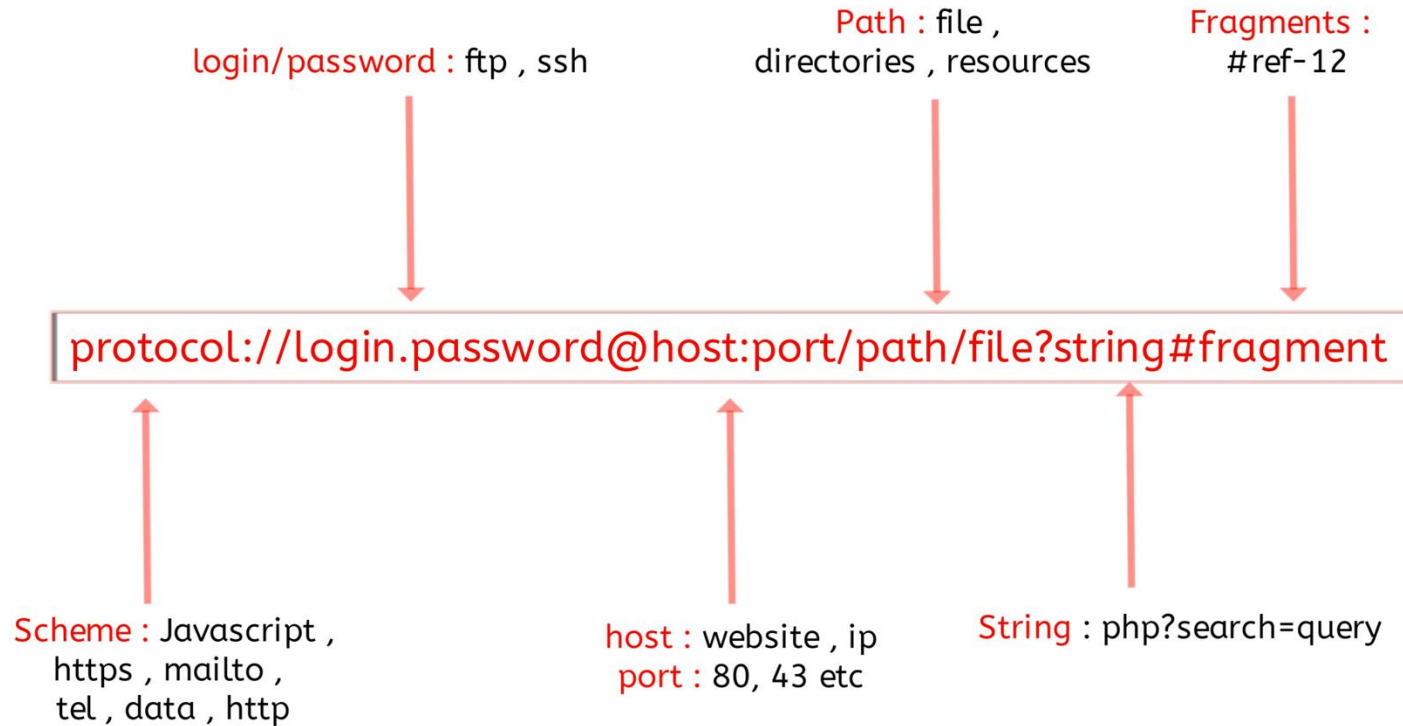
In next chapters, we'll create a webservice say user management with following functionalities –

| Sr.No. | URI                      | HTTP Method | POST body   | Result                      |
|--------|--------------------------|-------------|-------------|-----------------------------|
| 1      | /UserService/users       | GET         | empty       | Show list of all the users. |
| 2      | /UserService/addUser     | POST        | JSON String | Add details of new user.    |
| 3      | /UserService/getUser/:id | GET         | empty       | Show details of a user.     |

# REST and Resources



# URLs



jdbc:mysql://columbia-examples.ckkqqktwkcji.us-east-1.rds.amazonaws.com:3306

GET [http://localhost:5001/f23\\_imdb\\_clean/name\\_basics/nm0000158](http://localhost:5001/f23_imdb_clean/name_basics/nm0000158)

GET [http://localhost:5001/f23\\_imdb\\_clean/name\\_basics?deathYear=2023&birthyear=1960](http://localhost:5001/f23_imdb_clean/name_basics?deathYear=2023&birthyear=1960)

```
select * from f23_imdb_clean.name_basics where  
    deathYear=2023 AND birthyear=1960
```

PUT [http://localhost:5001/f23\\_imdb\\_clean/name\\_basics ?deathYear=2023&birthyear=1960](http://localhost:5001/f23_imdb_clean/name_basics ?deathYear=2023&birthyear=1960)

```
Body {'primaryName': 'Does not matter cause is dead.'}
```

```
update f23_imdb_clean.name_basics  
    set  
    where deathYear=2023 AND birthyear=1960
```

# *Locking Reference Material*



# Lock-Based Protocols

- A lock is a mechanism to control concurrent access to a data item
- Data items can be locked in two modes :
  1. **exclusive** (*X mode*). Data item can be both read as well as written. X-lock is requested using **lock-X** instruction.
  2. **shared** (*S mode*). Data item can only be read. S-lock is requested using **lock-S** instruction.
- Lock requests are made to concurrency-control manager. Transaction can proceed only after request is granted.



## Lock-Based Protocols (Cont.)

- **Lock-compatibility matrix**

|   | S     | X     |
|---|-------|-------|
| S | true  | false |
| X | false | false |

- A transaction may be granted a lock on an item if the requested lock is compatible with locks already held on the item by other transactions
- Any number of transactions can hold shared locks on an item,
- But if any transaction holds an exclusive on the item no other transaction may hold any lock on the item.



# Deadlock

- Consider the partial schedule

| $T_3$         | $T_4$         |
|---------------|---------------|
| lock-X( $B$ ) |               |
| read( $B$ )   |               |
| $B := B - 50$ |               |
| write( $B$ )  |               |
|               | lock-S( $A$ ) |
|               | read( $A$ )   |
|               | lock-S( $B$ ) |
| lock-X( $A$ ) |               |

- Neither  $T_3$  nor  $T_4$  can make progress — executing **lock-S( $B$ )** causes  $T_4$  to wait for  $T_3$  to release its lock on  $B$ , while executing **lock-X( $A$ )** causes  $T_3$  to wait for  $T_4$  to release its lock on  $A$ .
- Such a situation is called a **deadlock**.
  - To handle a deadlock one of  $T_3$  or  $T_4$  must be rolled back and its locks released.



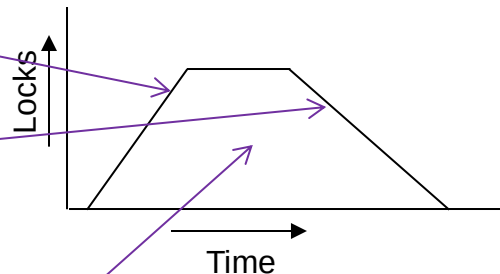
## Deadlock (Cont.)

- The potential for deadlock exists in most locking protocols. Deadlocks are a necessary evil.
- **Starvation** is also possible if concurrency control manager is badly designed. For example:
  - A transaction may be waiting for an X-lock on an item, while a sequence of other transactions request and are granted an S-lock on the same item.
  - The same transaction is repeatedly rolled back due to deadlocks.
- Concurrency control manager can be designed to prevent starvation.



# The Two-Phase Locking Protocol

- A protocol which ensures conflict-serializable schedules.
- Phase 1: **Growing Phase**
  - Transaction may obtain locks
  - Transaction may not release locks
- Phase 2: **Shrinking Phase**
  - Transaction may release locks
  - Transaction may not obtain locks
- The protocol assures serializability. It can be proved that the transactions can be serialized in the order of their **lock points** (i.e., the point where a transaction acquired its final lock).





## The Two-Phase Locking Protocol (Cont.)

- Two-phase locking *does not* ensure freedom from deadlocks
- Extensions to basic two-phase locking needed to ensure recoverability of freedom from cascading roll-back
  - **Strict two-phase locking:** a transaction must hold all its exclusive locks till it commits/aborts.
    - Ensures recoverability and avoids cascading roll-backs
  - **Rigorous two-phase locking:** a transaction must hold *all* locks till commit/abort.
    - Transactions can be serialized in the order in which they commit.
- Most databases implement rigorous two-phase locking, *but refer to it as simply two-phase locking*



# The Two-Phase Locking Protocol (Cont.)

- Two-phase locking is not a necessary condition for serializability
  - There are conflict serializable schedules that cannot be obtained if the two-phase locking protocol is used.
- In the absence of extra information (e.g., ordering of access to data), two-phase locking is necessary for conflict serializability *in the following sense*:
  - *Given a transaction  $T_i$  that does not follow two-phase locking, we can find a transaction  $T_j$  that uses two-phase locking, and a schedule for  $T_i$  and  $T_j$  that is not conflict serializable.*

| $T_1$         | $T_2$              |
|---------------|--------------------|
| lock-X( $B$ ) |                    |
| read( $B$ )   |                    |
| $B := B - 50$ |                    |
| write( $B$ )  |                    |
| unlock( $B$ ) |                    |
|               | lock-S( $A$ )      |
|               | read( $A$ )        |
|               | unlock( $A$ )      |
|               | lock-S( $B$ )      |
|               | read( $B$ )        |
|               | unlock( $B$ )      |
|               | display( $A + B$ ) |
| lock-X( $A$ ) |                    |
| read( $A$ )   |                    |
| $A := A + 50$ |                    |
| write( $A$ )  |                    |
| unlock( $A$ ) |                    |



# Locking Protocols

- Given a locking protocol (such as 2PL)
  - A schedule S is **legal** under a locking protocol if it can be generated by a set of transactions that follow the protocol
  - A protocol **ensures** serializability if all legal schedules under that protocol are serializable



# Lock Conversions

- Two-phase locking protocol with lock conversions:
  - Growing Phase:
    - can acquire a lock-S on item
    - can acquire a lock-X on item
    - can **convert** a lock-S to a lock-X (**upgrade**)
  - Shrinking Phase:
    - can release a lock-S
    - can release a lock-X
    - can convert a lock-X to a lock-S (**downgrade**)
- This protocol ensures serializability



# Automatic Acquisition of Locks

- A transaction  $T_i$  issues the standard read/write instruction, without explicit locking calls.
- The operation **read**( $D$ ) is processed as:
  - if**  $T_i$  has a lock on  $D$
  - then**
    - read( $D$ )
  - else begin**
    - if necessary wait until no other transaction has a **lock-X** on  $D$
    - grant  $T_i$  a **lock-S** on  $D$ ;
    - read( $D$ )
  - end**



## Automatic Acquisition of Locks (Cont.)

- The operation **write**( $D$ ) is processed as:  
if  $T_i$  has a **lock-X** on  $D$   
    **then**  
        write( $D$ )  
    **else begin**  
        if necessary wait until no other trans. has any lock on  $D$ ,  
        if  $T_i$  has a **lock-S** on  $D$   
            **then**  
                **upgrade** lock on  $D$  to **lock-X**  
            **else**  
                grant  $T_i$  a **lock-X** on  $D$   
        write( $D$ )  
    **end;**
- All locks are released after commit or abort



# Implementation of Locking

- A **lock manager** can be implemented as a separate process
- Transactions can send lock and unlock requests as messages
- The lock manager replies to a lock request by sending a lock grant messages (or a message asking the transaction to roll back, in case of a deadlock)
  - The requesting transaction waits until its request is answered
- The lock manager maintains an in-memory data-structure called a **lock table** to record granted locks and pending requests



# Deadlock Handling

- System is **deadlocked** if there is a set of transactions such that every transaction in the set is waiting for another transaction in the set.

| $T_3$         | $T_4$         |
|---------------|---------------|
| lock-X( $B$ ) |               |
| read( $B$ )   |               |
| $B := B - 50$ |               |
| write( $B$ )  |               |
|               | lock-S( $A$ ) |
|               | read( $A$ )   |
|               | lock-S( $B$ ) |
| lock-X( $A$ ) |               |



# Deadlock Handling

- **Deadlock prevention** protocols ensure that the system will *never* enter into a deadlock state. Some prevention strategies:
  - Require that each transaction locks all its data items before it begins execution (pre-declaration).
  - Impose partial ordering of all data items and require that a transaction can lock data items only in the order specified by the partial order (graph-based protocol).



# More Deadlock Prevention Strategies

- **wait-die** scheme — non-preemptive
  - Older transaction may wait for younger one to release data item.
  - Younger transactions never wait for older ones; they are rolled back instead.
  - A transaction may die several times before acquiring a lock
- **wound-wait** scheme — preemptive
  - Older transaction *wounds* (forces rollback) of younger transaction instead of waiting for it.
  - Younger transactions may wait for older ones.
  - Fewer rollbacks than *wait-die* scheme.
- In both schemes, a rolled back transactions is restarted with its original timestamp.
  - Ensures that older transactions have precedence over newer ones, and starvation is thus avoided.



## Deadlock prevention (Cont.)

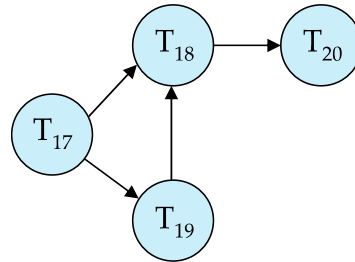
- **Timeout-Based Schemes:**

- A transaction waits for a lock only for a specified amount of time. After that, the wait times out and the transaction is rolled back.
- Ensures that deadlocks get resolved by timeout if they occur
- Simple to implement
- But may roll back transaction unnecessarily in absence of deadlock
  - Difficult to determine good value of the timeout interval.
- Starvation is also possible

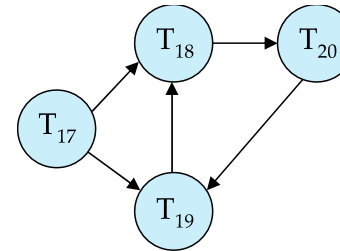


# Deadlock Detection

- **Wait-for graph**
  - *Vertices*: transactions
  - *Edge from  $T_i \rightarrow T_j$*  : if  $T_i$  is waiting for a lock held in conflicting mode by  $T_j$
- The system is in a deadlock state if and only if the wait-for graph has a cycle.
- Invoke a deadlock-detection algorithm periodically to look for cycles.



Wait-for graph without a cycle



Wait-for graph with a cycle

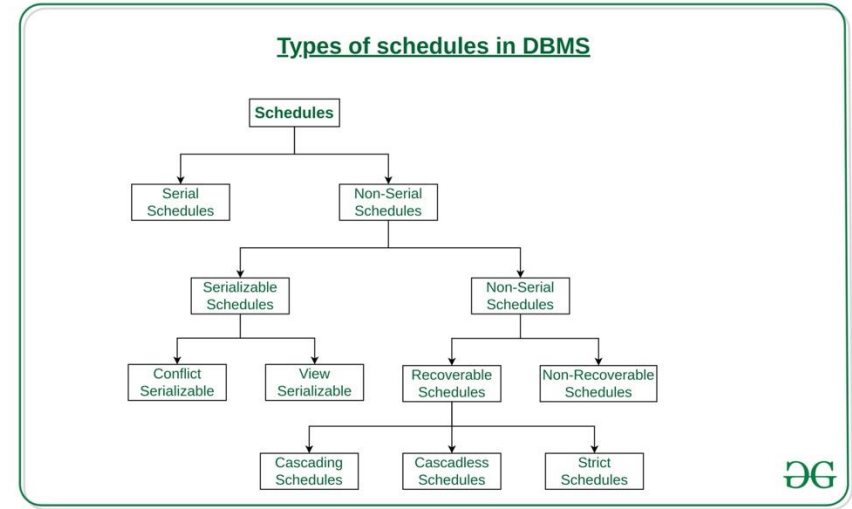
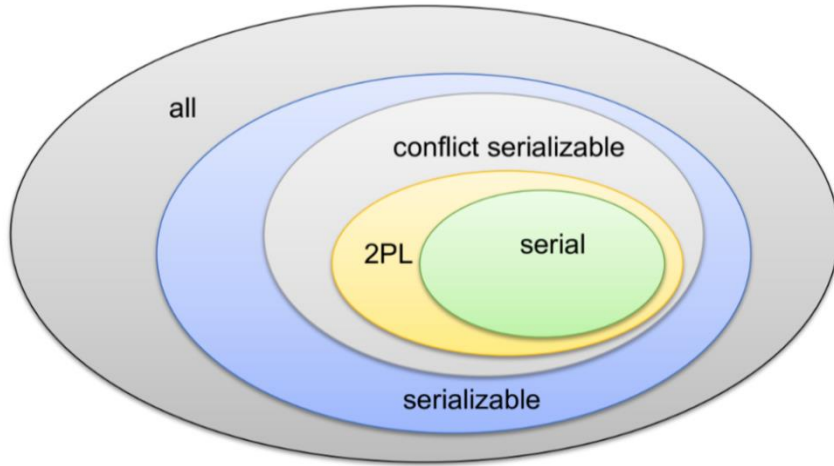


# Deadlock Recovery

- When deadlock is detected :
  - Some transaction will have to be rolled back (made a **victim**) to break deadlock cycle.
    - Select that transaction as victim that will incur minimum cost
  - Rollback -- determine how far to roll back transaction
    - **Total rollback**: Abort the transaction and then restart it.
    - **Partial rollback**: Roll back victim transaction only as far as necessary to release locks that another transaction in cycle is waiting for
- Starvation can happen (why?)
  - One solution: oldest transaction in the deadlock set is never chosen as victim

# Schedules, Serializable

# Schedules and Serializable



- We covered serial schedules, 2PL and serializable in the previous lecture.
- The types of schedules are much more complex.
- There are also forms of conflict management and lock types other than R/W.
- We will talk a little bit about them for awareness.
- These are academically interesting but do not come up in practice.



# Serializability

- **Basic Assumption** – Each transaction preserves database consistency.
- Thus, serial execution of a set of transactions preserves database consistency.
- A (possibly concurrent) schedule is serializable if it is equivalent to a serial schedule. Different forms of schedule equivalence give rise to the notions of:
  1. **Conflict serializability**
  2. **View serializability**



## ***Simplified view of transactions***

- We ignore operations other than **read** and **write** instructions
- We assume that transactions may perform arbitrary computations on data in local buffers in between reads and writes.
- Our simplified schedules consist of only **read** and **write** instructions.



# Conflicting Instructions

- Instructions  $I_i$  and  $I_j$  of transactions  $T_i$  and  $T_j$  respectively, **conflict** if and only if there exists some item  $Q$  accessed by both  $I_i$  and  $I_j$ , and at least one of these instructions wrote  $Q$ .
  1.  $I_i = \text{read}(Q)$ ,  $I_j = \text{read}(Q)$ .  $I_i$  and  $I_j$  don't conflict.
  2.  $I_i = \text{read}(Q)$ ,  $I_j = \text{write}(Q)$ . They conflict.
  3.  $I_i = \text{write}(Q)$ ,  $I_j = \text{read}(Q)$ . They conflict
  4.  $I_i = \text{write}(Q)$ ,  $I_j = \text{write}(Q)$ . They conflict
- Intuitively, a conflict between  $I_i$  and  $I_j$  forces a (logical) temporal order between them.
- If  $I_i$  and  $I_j$  are consecutive in a schedule and they do not conflict, their results would remain the same even if they had been interchanged in the schedule.



# Conflict Serializability

- If a schedule  $S$  can be transformed into a schedule  $S'$  by a series of swaps of non-conflicting instructions, we say that  $S$  and  $S'$  are **conflict equivalent**.
- We say that a schedule  $S$  is **conflict serializable** if it is conflict equivalent to a serial schedule



## Conflict Serializability (Cont.)

- Schedule 3 can be transformed into Schedule 6, a serial schedule where  $T_2$  follows  $T_1$ , by series of swaps of non-conflicting instructions. Therefore Schedule 3 is conflict serializable.

| $T_1$     | $T_2$     |
|-----------|-----------|
| read (A)  |           |
| write (A) |           |
|           | read (A)  |
|           | write (A) |
| read (B)  |           |
| write (B) |           |
|           | read (B)  |
|           | write (B) |

Schedule 3

| $T_1$     | $T_2$     |
|-----------|-----------|
| read (A)  |           |
| write (A) |           |
| read (B)  |           |
| write (B) |           |
|           | read (A)  |
|           | write (A) |
|           | read (B)  |
|           | write (B) |

Schedule 6



## Conflict Serializability (Cont.)

- Example of a schedule that is not conflict serializable:

| $T_3$         | $T_4$         |
|---------------|---------------|
| read ( $Q$ )  | write ( $Q$ ) |
| write ( $Q$ ) |               |

- We are unable to swap instructions in the above schedule to obtain either the serial schedule  $\langle T_3, T_4 \rangle$ , or the serial schedule  $\langle T_4, T_3 \rangle$ .



# View Serializability

- Let  $S$  and  $S'$  be two schedules with the same set of transactions.  $S$  and  $S'$  are **view equivalent** if the following three conditions are met, for each data item  $Q$ ,
  1. If in schedule  $S$ , transaction  $T_i$  reads the initial value of  $Q$ , then in schedule  $S'$  also transaction  $T_i$  must read the initial value of  $Q$ .
  2. If in schedule  $S$  transaction  $T_i$  executes **read**( $Q$ ), and that value was produced by transaction  $T_j$  (if any), then in schedule  $S'$  also transaction  $T_i$  must read the value of  $Q$  that was produced by the same **write**( $Q$ ) operation of transaction  $T_j$ .
  3. The transaction (if any) that performs the final **write**( $Q$ ) operation in schedule  $S$  must also perform the final **write**( $Q$ ) operation in schedule  $S'$ .
- As can be seen, view equivalence is also based purely on **reads** and **writes** alone.



## View Serializability (Cont.)

- A schedule  $S$  is **view serializable** if it is view equivalent to a serial schedule.
- Every conflict serializable schedule is also view serializable.
- Below is a schedule which is view-serializable but *not* conflict serializable.

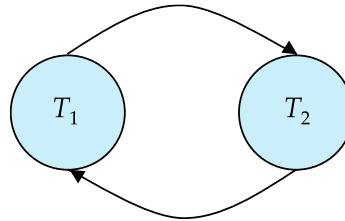
| $T_{27}$      | $T_{28}$      | $T_{29}$      |
|---------------|---------------|---------------|
| read ( $Q$ )  | write ( $Q$ ) |               |
| write ( $Q$ ) |               | write ( $Q$ ) |

- What serial schedule is above equivalent to?
- Every view serializable schedule that is not conflict serializable has **blind writes**.



# Testing for Serializability

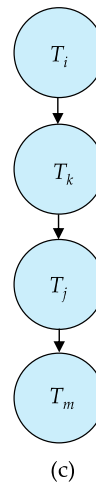
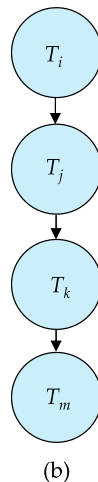
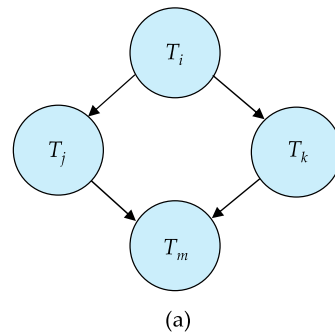
- Consider some schedule of a set of transactions  $T_1, T_2, \dots, T_n$
- **Precedence graph** — a direct graph where the vertices are the transactions (names).
- We draw an arc from  $T_i$  to  $T_j$  if the two transaction conflict, and  $T_i$  accessed the data item on which the conflict arose earlier.
- We may label the arc by the item that was accessed.
- Example of a precedence graph





# Test for Conflict Serializability

- A schedule is conflict serializable if and only if its precedence graph is acyclic.
- Cycle-detection algorithms exist which take order  $n^2$  time, where  $n$  is the number of vertices in the graph.
  - (Better algorithms take order  $n + e$  where  $e$  is the number of edges.)
- If precedence graph is acyclic, the serializability order can be obtained by a *topological sorting* of the graph.
  - This is a linear order consistent with the partial order of the graph.
  - For example, a serializability order for Schedule A would be  $T_5 \rightarrow T_1 \rightarrow T_3 \rightarrow T_2 \rightarrow T_4$ 
    - Are there others?





# Recoverable Schedules

Need to address the effect of transaction failures on concurrently running transactions.

- **Recoverable schedule** — if a transaction  $T_j$  reads a data item previously written by a transaction  $T_i$ , then the commit operation of  $T_i$  appears before the commit operation of  $T_j$ .
- The following schedule (Schedule 11) is not recoverable

| $T_8$     | $T_9$    |
|-----------|----------|
| read (A)  |          |
| write (A) |          |
|           | read (A) |
|           | commit   |
| read (B)  |          |

- If  $T_8$  should abort,  $T_9$  would have read (and possibly shown to the user) an inconsistent database state. Hence, database must ensure that schedules are recoverable.



# Cascading Rollbacks

- **Cascading rollback** – a single transaction failure leads to a series of transaction rollbacks. Consider the following schedule where none of the transactions has yet committed (so the schedule is recoverable)

| $T_{10}$                                      | $T_{11}$                      | $T_{12}$     |
|-----------------------------------------------|-------------------------------|--------------|
| read ( $A$ )<br>read ( $B$ )<br>write ( $A$ ) | read ( $A$ )<br>write ( $A$ ) |              |
| abort                                         |                               | read ( $A$ ) |

If  $T_{10}$  fails,  $T_{11}$  and  $T_{12}$  must also be rolled back.

- Can lead to the undoing of a significant amount of work



# Concurrency Control vs. Serializability Tests

- Concurrency-control protocols allow concurrent schedules, but ensure that the schedules are conflict/view serializable, and are recoverable and cascadeless .
- Concurrency control protocols (generally) do not examine the precedence graph as it is being created
  - Instead a protocol imposes a discipline that avoids non-serializable schedules.
  - We study such protocols in Chapter 16.
- Different concurrency control protocols provide different tradeoffs between the amount of concurrency they allow and the amount of overhead that they incur.
- Tests for serializability help us understand why a concurrency control protocol is correct.



# Weak Levels of Consistency

- Some applications are willing to live with weak levels of consistency, allowing schedules that are not serializable
  - E.g., a read-only transaction that wants to get an approximate total balance of all accounts
  - E.g., database statistics computed for query optimization can be approximate (why?)
  - Such transactions need not be serializable with respect to other transactions
- Tradeoff accuracy for performance



# Levels of Consistency in SQL-92

- **Serializable** — default
- **Repeatable read** — only committed records to be read.
  - Repeated reads of same record must return same value.
  - However, a transaction may not be serializable – it may find some records inserted by a transaction but not find others.
- **Read committed** — only committed records can be read.
  - Successive reads of record may return different (but committed) values.
- **Read uncommitted** — even uncommitted records may be read.

# ACID, CAP, BASE

# BASE

Eventually-consistent services are often classified as providing BASE (Basically Available, Soft state, Eventual consistency) semantics, in contrast to traditional ACID (Atomicity, Consistency, Isolation, Durability) guarantees. Rough definitions of each term in BASE:

- **Basically Available:** basic reading and writing operations are available as much as possible (using all nodes of a database cluster), but without any kind of consistency guarantees (the write may not persist after conflicts are reconciled, the read may not get the latest write)
- **Soft state:** without consistency guarantees, after some amount of time, we only have some probability of knowing the state, since it may not yet have converged
- **Eventually consistent:** If the system is functioning and we wait long enough after any given set of inputs, we will eventually be able to know what the state of the database is, and so any further reads will be consistent with our expectations

# ACID – BASE (Simplistic Comparison)

| ACID (relational)             | BASE (NoSQL)                        |
|-------------------------------|-------------------------------------|
| Strong consistency            | Weak consistency                    |
| Isolation                     | Last write wins (Or other strategy) |
| Transaction                   | Program managed                     |
| Robust database               | Simple database                     |
| Simple code (SQL)             | Complex code                        |
| Available and consistent      | Available and partition-tolerant    |
| Scale-up (limited)            | Scale-out (unlimited)               |
| Shared (disk, mem, proc etc.) | Nothing shared (parallellizable)    |

# CAP Theorem

- Consistency

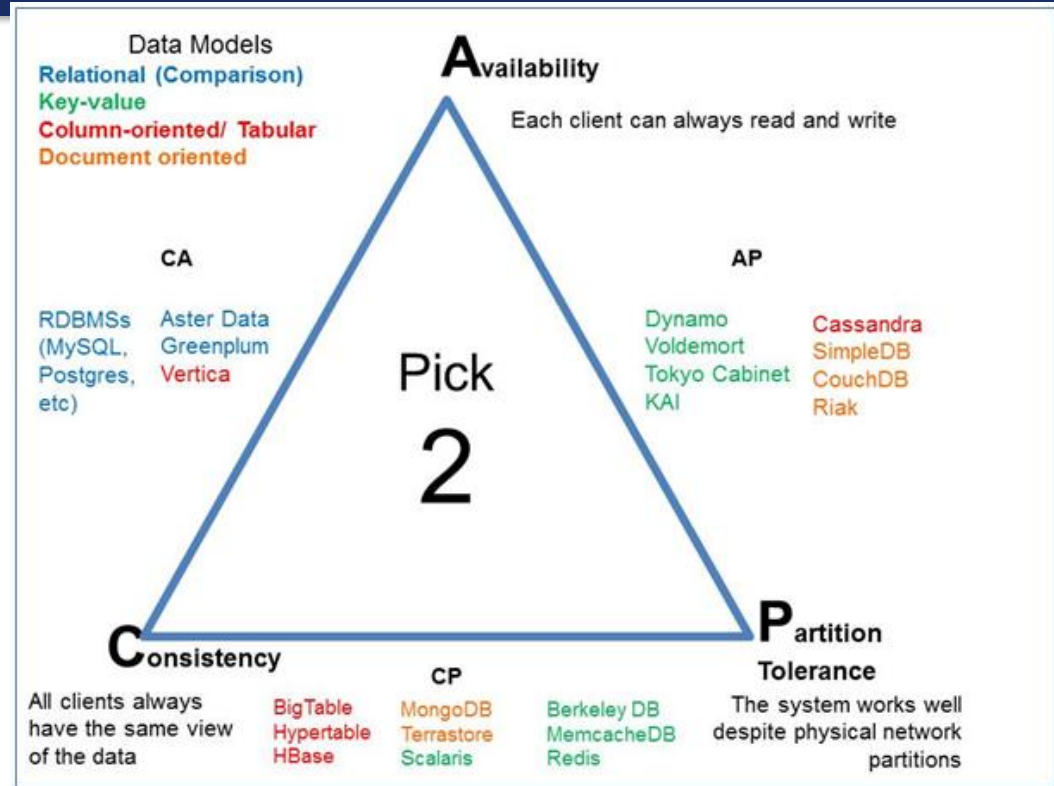
Every read receives the most recent write or an error.

- Availability

Every request receives a (non-error) response – without guarantee that it contains the most recent write.

- Partition Tolerance

The system continues to operate despite an arbitrary number of messages being dropped (or delayed) by the network between nodes



# Eventual Consistency

- Availability and scalability via
  - Multiple, replicated data stores.
  - Read goes to “any” replica.
  - PUT/POST/DELETE
    - Goes to any replica
    - Change propagate asynchronously
- GET may not see the latest value if the update has not propagated to the replica.
- There are several algorithms for conflict resolution
  - Detect and handle in application.
  - Clock/change vectors/version numbers
  - ... ..

